

Uso de LLMs para a previsibilidade de projetos de desenvolvimento de software

Rafael Galo Pugina

Trabalho de Conclusão de Curso
MBA em Inteligência Artificial e Big Data

UNIVERSIDADE DE SÃO PAULO

Instituto de Ciências Matemáticas e de Computação

Uso de LLMs para a previsibilidade
de projetos de desenvolvimento de
software

Rafael Galo Pugina

USP - São Carlos

2024

Rafael Galo Pugina

Uso de LLMs para a previsibilidade de projetos de desenvolvimento de software

Trabalho de conclusão de curso apresentado ao Departamento de Ciências de Computação do Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo - ICMC/USP, como parte dos requisitos para obtenção do título de Especialista em Inteligência Artificial e Big Data.

Área de concentração: Inteligência Artificial

Orientador: Prof. Dr. Lucas Correia Ribas e
Profa. Dra. Solange Oliveira Rezende

USP - São Carlos

2024

Ficha catalográfica elaborada pela Biblioteca Prof. Achille Bassi
e Seção Técnica de Informática, ICMC/USP,
com os dados inseridos pelo(a) autor(a)

P978u Pugina, Rafael
 Uso de LLMs para a previsibilidade de projetos
de desenvolvimento de software / Rafael Pugina;
orientador Lucas Correia Ribas; coorientador
Solange Oliveira Rezende. -- São Carlos, 2024.
 56 p.

 Trabalho de conclusão de curso (MBA em
Inteligência Artificial e Big Data) -- Instituto de
Ciências Matemáticas e de Computação, Universidade
de São Paulo, 2024.

 1. inteligência Artificial. 2. Machine Learning.
3. Coordenação de Projetos. 4. Estimativa de esforço.
I. Correia Ribas, Lucas , orient. II. Oliveira
Rezende, Solange, coorient. III. Título.

RESUMO

PUGINA, G. R. **Uso de LLMs para a previsibilidade de projetos de desenvolvimento de software.** 2024. 56 p. Trabalho de conclusão de curso (MBA em Inteligência Artificial e Big Data) – Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo, São Carlos, 2024.

Este trabalho investiga o uso de classificadores textuais e técnicas de processamento de linguagem natural (PLN) para otimizar a gestão de projetos de desenvolvimento de software. Dada a importância das estimativas de complexidade e tempo nas tarefas de planejamento, bem como seu impacto direto no andamento dos projetos, a pesquisa utiliza dados de tarefas previamente estimadas por equipes de desenvolvimento. Por meio de algoritmos de Aprendizado de Máquina e Modelos de Linguagem, o estudo busca classificar novas tarefas em contextos semelhantes, reduzindo a dependência do conhecimento especializado dos profissionais. A partir do software para gestão de projetos JIRA foram extraídos os dados relacionados a tarefas de trabalho e, após tratamento e análise, estes foram utilizados para a condução do treinamento e avaliação dos classificadores. Os resultados mostram que, nos testes propostos, a melhor performance foi obtida ao combinar o algoritmo Random Forest com o SBERT (Sentence Transformers) para análise de linguagem natural em descrições de tarefas. Este estudo oferece um ponto de partida para a implementação de soluções eficientes na classificação automática de tarefas, representando um recurso valioso para empresas que desejam agilizar as etapas de planejamento. O uso dessas técnicas pode adicionar um suporte baseado em dados para decisões que tradicionalmente se apoiam em intuição, aumentando a precisão e a velocidade das estimativas.

Palavras-chave: Processamento de Linguagem Natural; Aprendizado de Máquina; Gestão de Projetos; Classificação de Tarefas; Random-Forest; SBERT.

ABSTRACT

PUGINA, G. R. **Use of LLMs for Predictability in Software Development Projects.** 2024. 56 p. Final Paper (MBA in Artificial Intelligence and Big Data) – Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo, São Carlos, 2024.

This study investigates the use of text classifiers and natural language processing (NLP) techniques to optimize software development project management. Given the importance of complexity and time estimates in planning tasks, as well as their direct impact on project progress, the research utilizes data from tasks previously estimated by development teams. Through the use of Machine Learning algorithms and Language Models, the study aims to classify new tasks in similar contexts, reducing reliance on the specialized knowledge of professionals. Data related to work tasks were extracted from the project management software JIRA and, after processing and analysis, were used to train and evaluate the classifiers. The results show that in the proposed tests, the best performance was achieved by combining the Random Forest algorithm with SBERT (Sentence Transformers) for natural language analysis in task descriptions. This study provides a starting point for implementing efficient solutions in automatic task classification, representing a valuable resource for companies looking to streamline the planning stages. The use of these techniques can offer data-driven support for decisions that traditionally rely on intuition, increasing the accuracy and speed of estimates.

Keywords: Natural Language Processing; Machine Learning; Project Management; Task Classification; Random Forest; SBERT.

LISTA DE ILUSTRAÇÕES

Figura 1 – Pipeline de Classificação de texto	19
Figura 2 – Tokenização por Palavras e por Sentenças	22
Figura 3 – Diferença entre Stemming e Lemmatization	23
Figura 4 – Aprendizado de máquina tende a funcionar bem em dados balanceados	24
Figura 5 – Categorização de técnicas de Sub amostragem	25
Figura 6 – Vetorização e Vocabulário	26
Figura 7 – Classificação usando SVM	30
Figura 8 – Procedimento para classificação de texto supervisionada	31
Figura 9 – Linha do Tempo de lançamento de Grandes Modelos de Linguagem	32
Figura 10 – Fluxo de trabalho proposto	34
Figura 11 – Gráfico exibindo quantidade de valores únicos no campo Story Points.....	38
Figura 12 – Gráfico exibindo quantidade de valores únicos no campo gerado	39
Figura 13 – Gráfico exibindo quantidade de valores únicos no campo Complexity após random oversampling	41
Figura 14 – Representação gráfica do dataset após tratamento inicial	43
Figura 15 – Representação gráfica do dataset após união com embedding	46
Figura 16 – Comparativo dos resultados obtidos	50
Figura 17 – Matriz de Confusão para os testes de dataset tratado	51

LISTA DE TABELAS

Tabela 1 – Distribuição de características.....	36
Tabela 2 – Critérios para tratamento do valor Story Points.....	39
Tabela 3 – Resultado do tratamento de características.....	42
Tabela 4 – Encoding do campo Complexidade.....	43
Tabela 5 – Comparativo de resultado no dataset simplificado	45
Tabela 6 – Comparativo de resultado no dataset simplificado + summary.....	47
Tabela 7 – Comparativo de resultado no dataset simplificado + description.....	48
Tabela 8 – Resultados no dataset simplificado + summary + description.....	48
Tabela 9 – Resultados no dataset com o campo summary isolado.....	48
Tabela 10 – Resultados no dataset com o campo description isolado.....	49
Tabela 11 – Resultados no dataset com o campo description + campo summary.....	49

SUMÁRIO

1 INTRODUÇÃO	16
2 FUNDAMENTAÇÃO TEÓRICA	18
2.1 Dados de treinamento	19
2.2 Pré-processamento	21
2.2.1 Criação de Tokens.....	21
2.2.2 Stemming e Lemmatization	22
2.2.3 Balanceamento de dados	24
2.3 Extração de características.....	26
2.4 Algoritmos de Aprendizado de Máquina	27
2.4.1 Árvores de Decisão e Florestas Aleatórias.....	28
2.4.2 Naive Bayes	29
2.4.3 Máquinas de Vetores de Suporte (SVM)	29
2.4.4 Transformers e LLMs.....	30
2.4.5 SBERT - Sentence Transformers	32
2.5 Avaliação de Classificadores	33
3 METODOLOGIA	34
3.1 Métricas de Avaliação.....	35
3.2 Amostragem e Extração de Dados	35
3.3 Tratamento e Análise	37
3.3.1 Exploração e pré-processamento.....	37
3.3.2 Treinamento - Classificação/Predição usando algoritmos de AM	44
3.4 Discussão dos Resultados.....	49
4 CONCLUSÕES E TRABALHOS FUTUROS.....	52
REFERÊNCIAS.....	55

1 INTRODUÇÃO

A evolução da sociedade com a chegada do computador é irrefutável, e essa transformação está intrinsecamente ligada a uma revolução econômica notável. Conforme apontado pela Forbes (2024), sete das dez empresas mais valiosas do mundo, com base na capitalização de mercado em 2023, pertencem ao setor tecnológico, evidenciando o papel preponderante da tecnologia em nossa era.

Ao observar mais de perto esse fenômeno, percebe-se que, das inúmeras possibilidades proporcionadas pelo avanço tecnológico, destaca-se o desenvolvimento de software. Cada projeto nessa área compartilha um desafio comum: a tentativa de estimar o tempo, esforço e recursos necessários para atender às demandas específicas.

No universo do desenvolvimento de software, a projeção com precisão do tempo e dos recursos para a gestão de projetos é uma tarefa complexa e ininterrupta. Esse cenário dinâmico, caracterizado por inovações constantes, impõe desafios singulares à gestão de projetos, sobretudo para empresas que buscam liderança na criação de produtos inovadores. A abordagem convencional para antecipar o tempo e esforço requer uma descrição minuciosa do problema a ser solucionado e a consulta a especialistas com conhecimento prévio. No entanto, essa prática, por vezes, exige várias iterações sobre o problema, prolongando o tempo necessário para a estimativa. Quanto maior o projeto ou tarefa, maior a margem de erro; quanto menor o conhecimento aprofundado sobre a demanda, maior a incerteza na estimativa.

Estimar o tempo e esforço em projetos de software é uma necessidade crucial por diversos motivos. Além de entender se as expectativas do cliente podem ser atendidas, essas estimativas são fundamentais para gerar engajamento e comprometimento da equipe, bem como para viabilizar a execução do projeto como um todo. Independentemente do motivo, a gestão efetiva de um projeto sempre demandará, de alguma forma, abordagens relacionadas a essas estimativas.

Diversas linhas de pesquisa têm sido exploradas na aplicação de técnicas de Inteligência Artificial (IA) para aprimorar a gestão de projetos. Essas abordagens englobam uma variedade de áreas, desde a avaliação de candidatos, como conduzido por Menon e Rahulnath (2017) em seu estudo sobre seleção de pessoal, até a modelagem e mitigação de riscos, investigada por Hu et al. (2016), assim como por Ko e Cheng (2007). Além disso, essas técnicas têm sido aplicadas na direção das linhas de previsões e estimativas, conforme evidenciado no trabalho de Jin et al. (2016), nos quais é apresentado um modelo matemático para a integração de planejamento e agendamento de processos, considerando a incerteza inerente ao tempo de processamento. Essa

diversidade de abordagens destaca a versatilidade e a aplicabilidade da IA no contexto dinâmico da gestão de projetos.

Projetos que adotam metodologias ágeis buscam fundamentar seus planejamentos e processos em dados relacionados a eventos anteriores da equipe ou do projeto. Nesse contexto, a Inteligência Artificial emerge com recursos especialmente valiosos, conforme mencionado por Dam et al. (2019) em sua pesquisa sobre gestão de projetos ágeis potencializada pela IA.

Este trabalho tem como objetivo utilizar técnicas de aprendizado de máquina e processamento de texto para propor uma solução estratégica visando auxiliar gestores de projetos e equipes de desenvolvimento de software a estimar esforços de trabalho, seja em tempo ou pontos relativos. Para tanto, será utilizado conhecimento e dados relacionados aos projetos em andamento ou finalizado, velocidade atual de entrega e contexto tecnológico. Diante dos desafios previamente mencionados, essa solução promete ser particularmente valiosa na obtenção de estimativas preliminares nas fases iniciais do desenvolvimento, em que a viabilidade do projeto está em jogo. Além disso, destaca-se como uma ferramenta significativa durante a análise de uma demanda por especialistas ou equipes de desenvolvimento, facilitando discussões construtivas diante de possíveis discordâncias entre as opiniões profissionais e os valores gerados automaticamente.

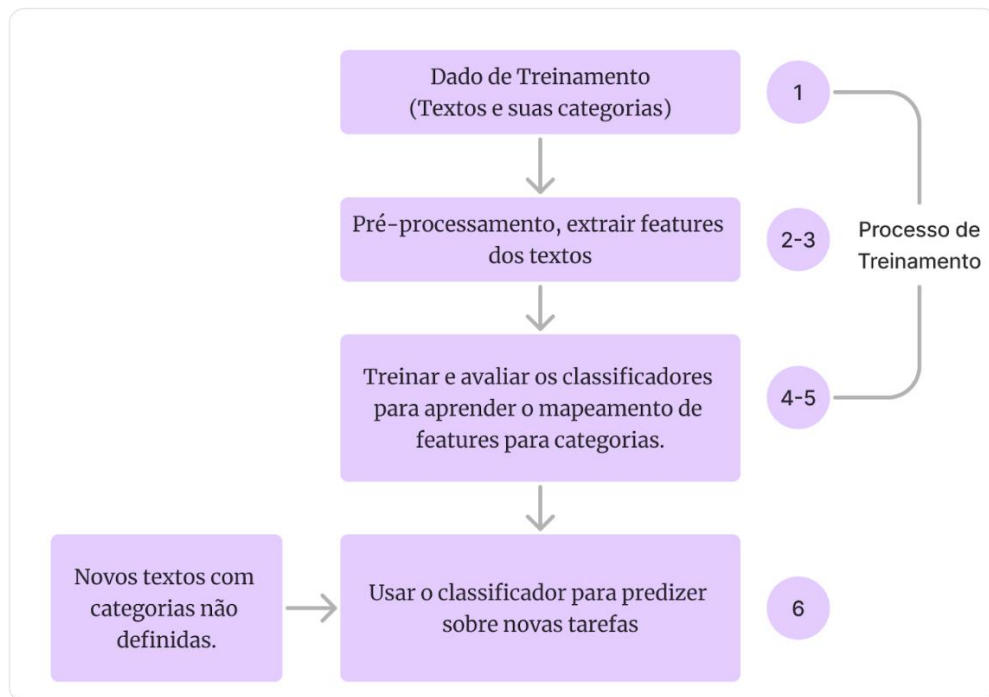
2 FUNDAMENTAÇÃO TEÓRICA

Classificação, de forma geral, dentro do contexto de aprendizado de máquina, se resume em categorizar uma instância de dado, seja esse dado formado por texto, fala, imagem ou número, em uma ou mais classes anteriormente estabelecidas. O texto, em especial, pode ser de tamanho variado, desde um único caractere até documentos inteiros. O desafio é aprender com classificações anteriores e, ao ser proposto um texto completamente novo, sem dado de classificação, ser capaz de chegar em uma classificação adequada.

A classificação de textos é uma das tarefas mais comuns em Processamento de Língua Natural. Sua atuação varia muito e se faz presente em diversos tipos de aplicações, algumas muito comuns como categorizar um ticket de suporte baseado na língua em que ele foi escrito ou ainda segregar automaticamente emails que são considerados lixo eletrônico, levando em consideração seu conteúdo. Nos dias de hoje muito do estudo se direciona a resolver problemas que emergem com as necessidades criadas pela própria tecnologia, como por exemplo identificar emoções de um tweet ou comentário de rede social em um produto de sua empresa (TUNSTALL, 2022).

Para se fazer a classificação costuma-se seguir uma ordem específica de etapas, essas são apresentadas na Figura 1, e suas etapas são: 1. Criar ou coletar um dataset apropriado para a tarefa. 2. Dividir o dataset em treinamento e teste. 3. Extrair os vetores característicos. 4. Utilizar os vetores característicos para treinar o classificador a encontrar seus rótulos. 5. Por meio de métricas, definir o desempenho do modelo. 6. Utilizar o modelo para resolver problemas do mundo real e monitorar seu desempenho.

Figura 1 – Pipeline de Classificação de texto



Fonte: Baseado em (VAJJALA et al., 2020).

2.1 Dados de treinamento

Como explicado brevemente, os dados de treinamento são formados por um conjunto de dados que será utilizado para treinar um dado modelo, é importante que esse conjunto de dados, além do texto de análise com dados preferencialmente relacionados ao conteúdo que provavelmente o modelo será submetido quando aplicado ao problema real, também contenha a classificação ideal já definida. Isso serve para que o modelo tenha referências de aprendizado.

À primeira vista essa etapa pode parecer simples, mas na verdade ela pode se tornar bem trabalhosa e ser o fator mais importante para o sucesso do projeto. Em um cenário perfeito, sempre se tem disponível um conjunto de dados ideal, já classificado, com milhares e talvez até milhões de exemplos. Nesse caso não seria necessário nos preocupar-se demais com a coleta dos dados, porém a verdade é que raramente alguém se encontra nessa situação. Existem sim algumas situações em que os dados já estão praticamente prontos, mas a realidade é que na maioria dos projetos de Inteligência Artificial aplicada à indústria, tem-se muito pouco (ou nenhum) dado preparado, e cabe aos analistas tratar esse dado em termos de qualidade e quantidade.

No que diz respeito a adquirir dados, existem diversas opções quando o cenário atual é mais realista do que o então descrito mundo perfeito. Pode-se utilizar de *datasets* públicos, que podem ser adquiridos por meio de listas publicadas na web, como a mantida no *github* de Iderhoff (2024), ou pelo Google Dataset Search (2024).

Quando não encontrado um *dataset* de contexto similar ao trabalho em questão, outra opção é fazer uso de uma técnica conhecida como *scraping* que consiste em coletar dados diretamente da internet, um exemplo poderia ser buscar comentários negativos diretamente em sites de reclamação, e positivos no mural de uma empresa de atuação similar, ou até mesmo avaliações de compras em sites de e-commerce. Para muitas aplicações, o *scraping* se torna ineficaz pois não possui dados com palavras chaves relacionadas a empresa ou negócio, o que inviabiliza o uso da técnica.

Uma terceira forma comumente utilizada por grandes empresas é por meio de intervenção no próprio produto. Essa técnica tem sido explorada fortemente pelas grandes empresas de tecnologia, que já coletam os dados de seus usuários há muito tempo e estão sempre buscando encontrar formas melhores e mais eficientes de fazê-lo. Tome como exemplo um site de vendas online em que coletar os dados de navegação de cada usuário se torna extremamente valioso quando o intuito é sugerir novos produtos similares a este próprio usuários, ou para usuários com localidade e outras características similares, e esse *dataset* foi gerado por meio da utilização do produto.

Após obter o conjunto de dados inicial, surge um desafio crucial relacionado à quantidade de entradas. Existem diversas técnicas que podem ser empregadas para aprimorar essa característica do conjunto de dados. Entre uma ampla variedade de abordagens, algumas podem ser consideradas, como o *back translation*. Essa técnica envolve a tradução de um texto para uma língua diferente da original, seguida pela re-tradução para o idioma inicial, utilizando ferramentas como o Google Tradutor ou similares. Outra estratégia conhecida é a substituição de entidades, tais como locais, organizações e nomes, por outras entidades da mesma categoria. Além disso, a troca de palavras por sinônimos, alterando o texto sem modificar seu sentido, pode ser uma abordagem viável, utilizando uma quantidade previamente definida de substituições. Dado que muitos conjuntos de dados podem conter erros ortográficos e gramaticais, com base no contexto de onde esses dados foram extraídos, outra técnica empregada é a adição proposital de ruído nos dados, simulando tais erros.

Para todas as opções mencionadas, bem como outras possíveis estratégias, o objetivo final é obter um conjunto de dados preparado para a próxima etapa: a limpeza dos dados, que consiste em extrair o texto puro a partir dos dados de entrada, dados esses que originalmente

podem conter informações não textuais dependendo do formato que foram gerados. Se o padrão de coleta for HTML, por exemplo, faz parte da etapa de limpeza remover as tags específicas, deixando apenas o texto pré-formatado para o processamento (VAJJALA et al., 2020).

2.2 Pré-processamento

Partindo do ponto após a limpeza, pode-se pensar que o *dataset* está pronto para ser utilizado, e que não há mais trabalho a ser feito sobre o conteúdo textual em questão. A verdade é que se precisa de algumas especificações a mais para que o treinamento funcione corretamente. Em especial, quando se busca pelo processamento de língua natural, os algoritmos precisam de uma preparação de palavras mínima. As tarefas a seguir são algumas de muitas daquelas que comumente pode-se recorrer na tarefa de pré-processamento dos dados, que é uma etapa importante do pipeline onde os dados de entrada e os modelos a serem usados são de grande importância na decisão de quais caminhos tomar.

2.2.1 Criação de Tokens

A *Tokenização* de Palavras é um processo em que se busca dividir um determinado texto, em sua forma original, em pequenos blocos de palavras ou sentenças. Pensando na divisão por palavras, por exemplo, localiza-se os limites destas, ou seja, os pontos onde uma palavra termina e a outra começa, uma forma de fazer isso é considerar o caráter em branco “espaço” para fazer essa distinção e separação, transformando assim uma grande *String* em um vetor de *Strings*.

A *Tokenização* de Sentenças, por sua vez, é o processo de determinar unidades de processamento mais longas, que pode ser constituído de uma ou mais palavras. Na maioria dos idiomas escritos os sinais de pontuação são ótimos indicadores desses limites, pois é justamente ocorre, mas isso não é regra, como em abreviações, por exemplo. Ambos os termos se referem à tarefa de determinar como um texto deve ser dividido, e são etapas preliminares do pré-processamento (PALMER, 2000).

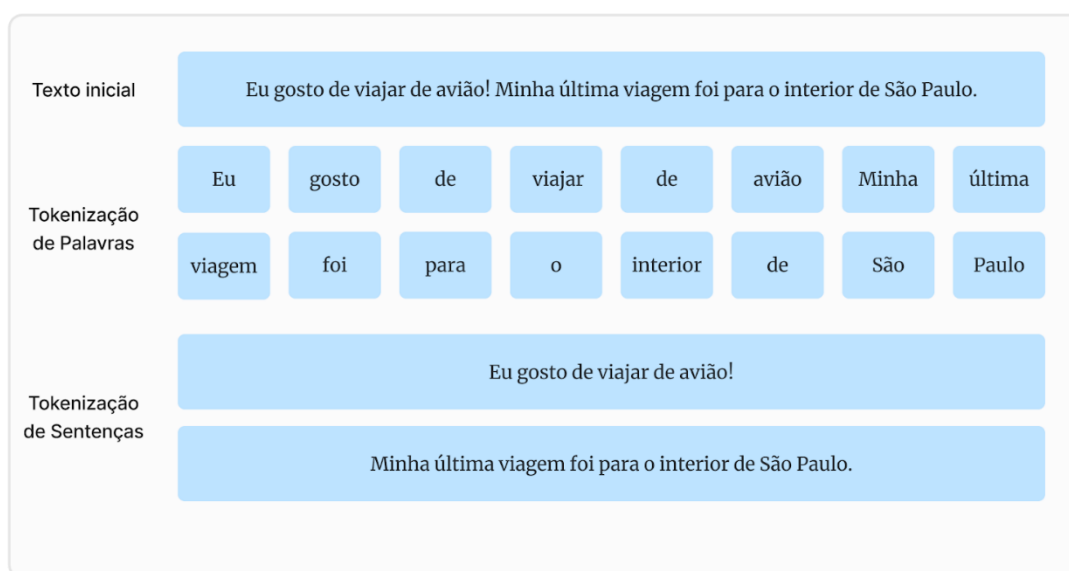
Escolher entre *Tokenização* de Palavras ou de Sentenças dependerá muito do que se planeja fazer após esse processamento, qual a próxima etapa do pipeline e qual o formato esperado no próximo passo.

A importância e o valor da *Tokenização* pode ser identificada de diversas maneiras, como preparar o texto para uma contagem de palavras, ajudando a identificar a importância de

cada uma dentro do texto por meio de sua ocorrência, pode-se também fazer dessa etapa um predecessor a vetorização do texto, com a intenção de transformar cada uma das palavras em um valor inteiro, representando o texto todo posteriormente de forma vetorial, muito útil para os modelos de aprendizado de máquina, que trabalham melhor com números do que caracteres diretamente.

Existem muitas técnicas diferentes usadas para essa tarefa, desde as mais simples como utilizar bibliotecas já inclusas em linguagens de programação para fazer a divisão, exemplo da função "split()" do Python, mas também as mais elaboradas e customizadas, levando em consideração o texto em que se pretende trabalhar e suas peculiaridades.

Figura 2 – Tokenização por Palavras e por Sentenças



A Tokenização é o processo de dividir a sequência de caracteres no texto, localizando os limites das palavras, ou seja, os pontos onde uma palavra termina e a outra começa. Chama-se de token essa unidade, que se resume a uma palavra.

2.2.2 Stemming e Lemmatization

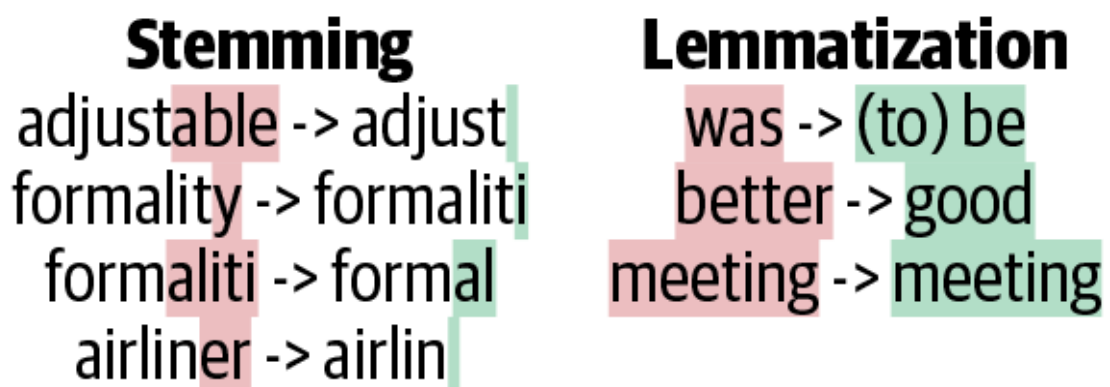
Outro processo comum no pré-processamento é o de reduzir palavras a sua forma base ou raiz, isso para tentar diminuir o vocabulário do dataset e tornar o processamento mais eficiente. veja por exemplo as palavras casa, casarão, casinha e casebre, todas podem ser reduzidas para sua forma raiz “casa” sem modificar sua essência significativa. Essa redução é

feita por uma biblioteca desenvolvida para o idioma do texto ou uma série de regras linguísticas específicas para o conteúdo.

Essas regras e bibliotecas dependem diretamente da língua em que o texto foi escrito, que por sua vez podem vir a ser complexas e numerosas, em alguns casos o resultado pode vir a não ser a base correta, linguisticamente falando, ou mesmo não fazer sentido, mas seu valor é agregado posteriormente nas etapas de processamento e análise estatística, esse processo é conhecido por stemming ou radicalização.

Muitas vezes o processo de stemming é acompanhado por um outro, que traz algumas semelhanças, a Lemmatization ou a Lematização. Se a primeira buscava ajustar palavras ao seu radical, a segunda busca sua forma lema, que é a forma como são apresentadas nos tradicionais dicionários.

Figura 3 – Diferença entre Stemming e Lemmatization



Fonte: VAJJALA et al. (2020).

Existem também alguns outros passos frequentes dentro do pré-processamento, como a remoção das stop words, um processo que visa remover palavras que são irrelevantes para o processamento do texto, remoção de pontuação ou caracteres especiais ou mesmo a transformação de caixa alta para caixa baixa (lowercasing).

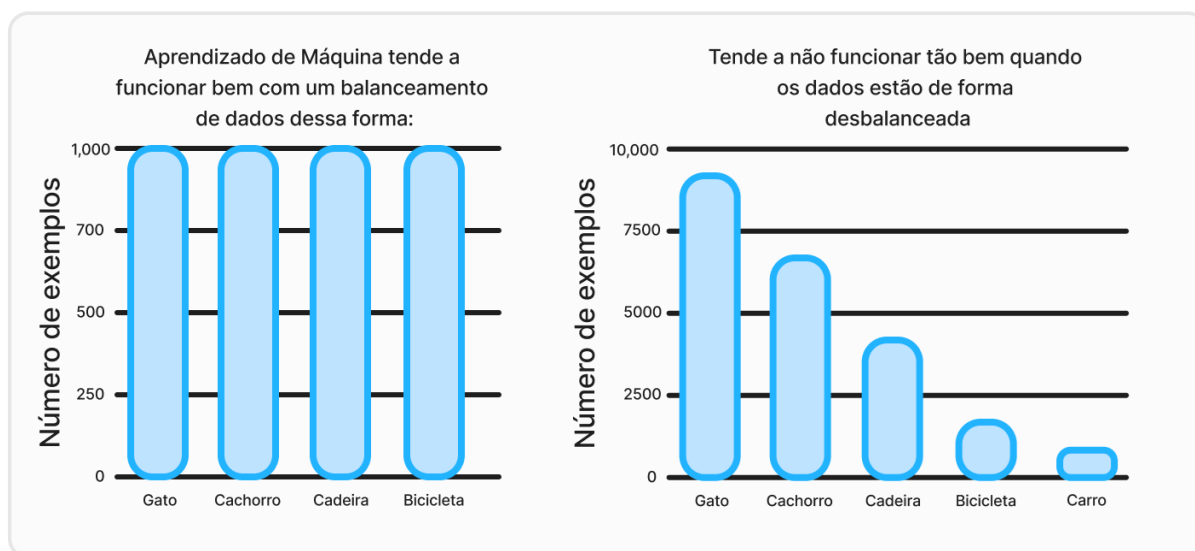
Além do stemming, é comum encontrar a técnica chamada de lematização, que, à primeira vista, pode parecer similar, mas são abordagens diferentes. A lematização tem como objetivo reduzir palavras flexionadas ao seu formato de dicionário, levando em consideração a morfologia das palavras. Por exemplo, a palavra 'cantar' pode ser considerada a forma base para 'canto', 'cantoria', 'cantei', 'cantamos' e 'cantaram'. Embora seja uma técnica mais robusta e

precisa, a lematização vem acompanhada de custos computacionais e complexidade mais elevados, além de que, para ser modelada, exige maiores conhecimentos linguísticos.

2.2.3 Balanceamento de dados

Durante os estudos acadêmicos, é comum encontrar dados destinados a aprendizado que são preparados de forma já balanceada, isso quer dizer que a classe objetivo tem uma distribuição igualitária (ou quase) de possibilidades. Um exemplo seria um dataset que busca uma classificação binária entre comentários positivos e negativos sobre uma empresa em uma rede social, onde se tem no total 5000 comentários, sendo que 2600 são positivos e 2400 são negativos. Para esse caso exemplo, os dados estão balanceados, tem-se quase a mesma quantidade de comentários, portanto quando um modelo for submetido a treinamento, ele tem uma quantidade similar de exemplos para aprender sobre o comportamento de ambas as classes. A Figura 4 ilustra a diferença entre dados balanceados e desbalanceados, e aponta onde o aprendizado de máquina tende a ser mais eficiente. Mas vale ressaltar que a representatividade dos dados é o fator que normalmente exerce maior peso.

Figura 4 – Aprendizado de Máquina tende a funcionar bem em dados balanceados



Fonte: Andrew Ng. (2020).

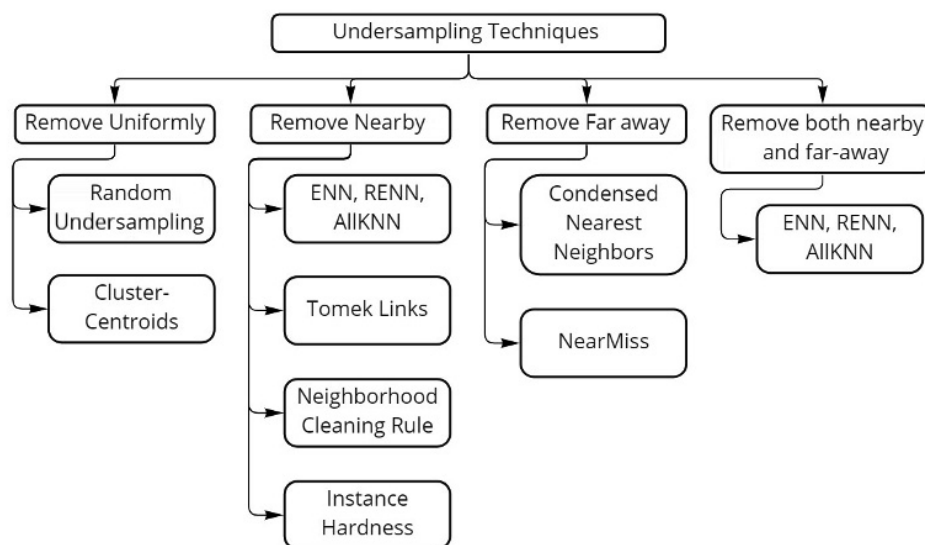
E cenários do mundo real, o normal é que os dados estejam desbalanceados, além de que em uma considerável porção desses casos, a classe de maior importância é justamente a minoritária. Um exemplo, relacionado à questão da importância, seria onde se dados de

diagnóstico médico para pacientes com câncer, a relação entre o positivo e negativo pode ser muito baixa, por exemplo 1:1000, mas o interesse em diagnosticar corretamente pessoas que têm câncer como positivo é muito maior do que de diagnosticar pessoas que não possuem como negativo, ou seja, o interesse maior é justamente pela porção onde se tem uma quantidade menor de exemplos (HUYEN., 2022).

O processo de amostragem tem objetivo de tratar algumas questões, dentre elas classes desbalanceadas, este pode acontecer no pré-processamento ou em etapas de iteração do pipeline, tem o intuito de balancear os dados e facilitar o trabalho dos modelos de aprendizado de máquina.

Dentre algumas técnicas para tentar contornar o desbalanceamento, tem-se a Sobre amostragem (*Oversampling*) e a Sub amostragem (*Undersampling*). O primeiro é um método que busca aumentar a quantidade de entradas de uma determinada classe, isso é feito usando algumas técnicas específicas, como o *Random Oversampling*, o SMOTE (*Synthetic Minority Oversampling Technique*) ou suas variações e o ADASYN (*Adaptive Synthetic Sampling*), é normalmente escolhido quando não se tem exemplos suficientes da classe minoritária, deixando a possibilidade de o modelo simplesmente ignorar a classe na hora da classificação. Para a segunda, como consiste em remover entradas da base analisada, é importante responder algumas perguntas, visto que essa remoção pode ajudar ou prejudicar a definição dos limites de classificação, a Figura 5 ilustra algumas das possíveis escolhas.

Figura 5 – Categorização de técnicas de Sub amostragem



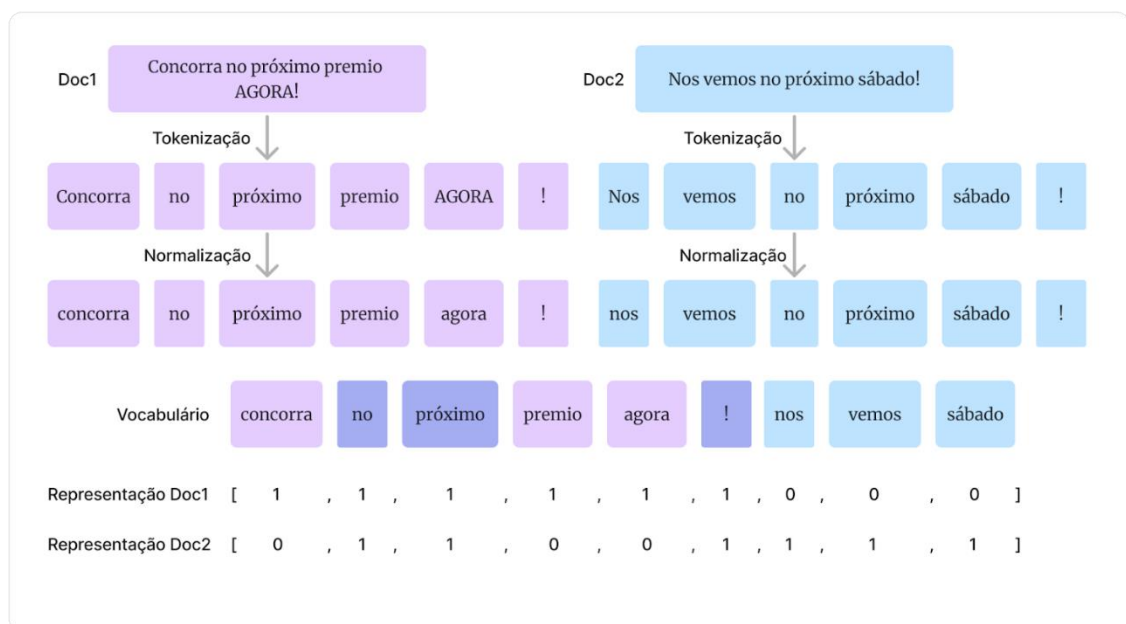
2.3 Extração de características

Uma vez que o trabalho de separar e extrair as palavras está completo, parte-se para a etapa de extrair as características do *dataset*, amplamente referenciado como *feature extraction*, esse consiste em converter os textos em representações numéricas. Para atingir tal objetivo, existem algumas técnicas, como a “*Bag of Words*” ou saco de palavras e TF-IDF (*Term Frequency-Inverse Document Frequency*).

Na primeira, o objetivo é representar de forma numérica e vetorial as palavras contidas no texto, com o intuito de tornar mais eficiente o trabalho de processamento dos algoritmos de aprendizado de máquina.

Para exemplificar, pense na tarefa de detectar conteúdo de *spam* das ferramentas de e-mail, simplificando para fins de ilustração, nossos e-mails têm em seu conteúdo apenas uma frase simples, a Figura 6 apresenta uma representação dos passos discutidos, e a conversão das palavras para sua representação vetorial após a extração das características. Na imagem é possível visualizar que cada um dos documentos é convertido em representações vetoriais, partindo de uma base que é formada por um vocabulário que contém todas as palavras de ambos os documentos, porém sem repetir representações.

Figura 6 – Vetorização e Vocabulário



Fonte: Baseado em KOCHMMAR. (2022).

O intuito dessa representação é justamente comparar os vetores representados e definir, baseado em um nível de expectativa, se esses vetores podem ser considerados similares o suficiente para serem definidos como parte do mesmo “bag” ou saco, que pode ser relacionado com a definição de classe discutida anteriormente.

A principal diferença para a abordagem TF-IDF é que, diferente do BoW que trata todas as palavras como igualmente importantes, esta tenta quantificar e levar em consideração a importância individual de cada uma das palavras em relação às outras do corpus.

Para entender a abordagem, imagine que uma palavra “p” apareça constantemente em um determinado documento “d”, porém ao olhar para outros documentos do corpus, essa palavra já não aparece com tanta frequência assim, conclui-se, portanto, que a palavra “p” tem grande importância no “d”. Ao mesmo tempo que a importância da palavra aumenta, dado sua frequência em um documento, ela deve também aumentar de forma proporcional a sua ausência em outros documentos. Matematicamente, essa equação é capturada usando dois valores, a Frequência do Termo (Document Frequency - DF) e a Frequência Inversa do Documento (Inverse Document Frequency - IDF).

$$TF(p, d) = \frac{(\text{Número de ocorrências da palavra } p \text{ no documento } d)}{(\text{Total de ocorrências da palavra } p \text{ no documento } d)}$$

$$IDF(p) = \log e \frac{(\text{Número total de documentos no corpus})}{(\text{Número de documentos contendo a palavra } p)}$$

Como se pode imaginar, a combinação das duas equações é o que consiste a abordagem TF-IDF, em particular o produto dos dois termos, ou seja, $TF * IDF$ = pontuação que será usada para considerar a importância de cada palavra no corpus.

Assim como o BoW, a classificação das palavras pode ser feita utilizando medidas de similaridade vetorial como distância euclidiana ou similaridade de cosseno (VAJJALA et al., 2020).

2.4 Algoritmos de Aprendizado de Máquina

Definido como um subcampo da Inteligência Artificial, o aprendizado de máquina é a área que se concentra em buscar uma forma de fazer com que as máquinas simulem o comportamento humano de aprender com experiências e informações, faz isso por meio do

desenvolvimento de algoritmos e modelos para este fim, identificando padrões e fazendo previsões sem serem programados especificamente para a tarefa em questão (GÉRON, 2017).

Para saber se o modelo se sairá bem na tarefa de classificação, deve-se colocar o modelo a prova com novos casos. Uma abordagem poderia ser colocá-lo em produção diretamente na aplicação para qual ele foi projetado. Isso pode funcionar muito bem, porém se o modelo se provar ineficiente e trazer resultados muito ruins, é bem provável que as reclamações chegarão, tornando esse plano uma má ideia.

De maneira mais sensata, uma alternativa seria dividir os dados que possuímos em duas partes distintas: uma de treinamento e outra de teste. A ideia por trás disso é empregar os dados de treinamento, os quais incluem as respostas corretas das classificações, para instruir o modelo na tarefa de classificação. Por sua vez, as respostas são ocultadas nos dados de teste, o que desafia o modelo a prever as classificações por conta própria. Dessa forma, ao comparar as previsões com as respostas reais, pode-se estimar o erro de generalização que o modelo apresentará quando confrontado com dados de produção previamente não observados.

Com esse resultado em mãos, é possível avaliar se o modelo está desempenhando satisfatoriamente ou se está sofrendo de *overfitting*, um fenômeno em que o modelo se ajusta excessivamente aos dados de treinamento, resultando em um desempenho ruim com novos dados.

Existem diversas abordagens disponíveis para a tarefa de classificação, que podem incluir modelos baseados em regras, aprendizado supervisionado ou aprendizado profundo. A escolha da técnica mais adequada depende amplamente do cenário específico e dos objetivos em termos de precisão, desempenho, recursos computacionais, entre outros fatores. A seguir, são descritas brevemente algumas das técnicas mais comumente utilizadas.

2.4.1 Árvores de Decisão e Florestas Aleatórias

São técnicas de aprendizado supervisionado, baseadas em decisões sequenciais, em que se divide os dados em subconjuntos. A estrutura o algoritmo de árvores de decisão busca se assemelhar a de uma árvore real, contendo raiz, que inicialmente é o *dataset*, os nós internos que representam os testes de determinados atributos, galhos que representam os resultados de um teste e as folhas, que representam uma atribuição de classe. A Floresta Aleatória é um modelo que se baseia em agregar múltiplas árvores de decisão. Ambas são muito populares devido ao bom desempenho em relação ao *overfitting* bem como à resiliência em relação a dados ruidosos (HASTIE; TIBSHIRANI; FRIEDMAN, 2009).

2.4.2 Naive Bayes

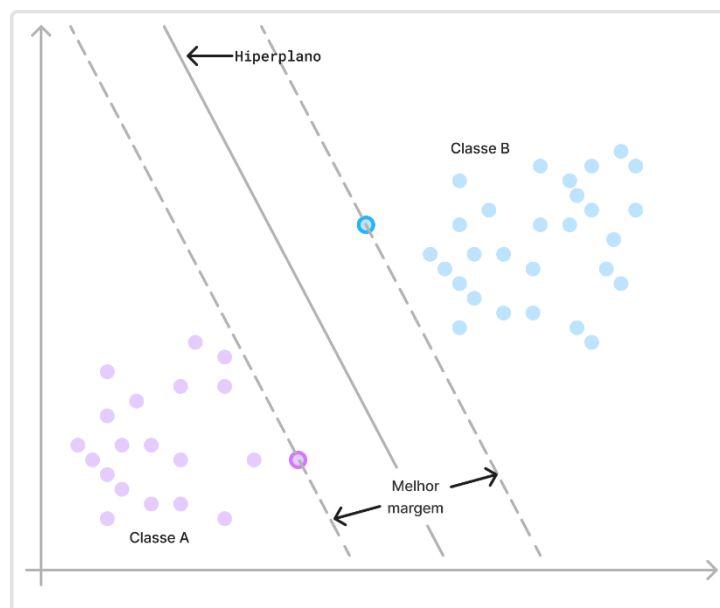
É um modelo probabilístico amplamente utilizado, também conhecido por sua facilidade de implementação em relação a sua eficiência. Sua classificação se dá por meio da aplicação do teorema de Bayes, assumindo uma forte interdependência entre as características. Essa interdependência condicional pode se tornar um problema em conjunto de dados altamente correlacionados, sendo violada muitas vezes na prática e tratada diversamente na literatura como não realista, mesmo assim o Naive Bayes se prova surpreendentemente eficiente em muitos cenários e traz resultados competitivos com relação a outros modelos (WEBB et al., 2010).

2.4.3 Máquinas de Vetores de Suporte (SVM)

São modelos que inicialmente foram destinados para classificação binária, mas posteriormente estendidos para trabalhar com multi-casos, foram inventadas no início dos anos 60 e provam ser úteis até hoje. São classificadores discriminativos que, diferente de outros do mesmo tipo, buscam um chamado hiperplano ótimo para separar as classes olhando para uma máxima margem possível. (GÉRON., 2017)

O hiperplano, em um cenário de classificação binária, pode ser definido como uma linha traçada entre a representação gráfica de cada classe, e o objetivo da SVM é justamente encontrar essa linha com a maior margem possível entre as classes, como mostrado na Figura 7. Após definida a linha, para classificar novas entradas deve-se então buscar definir em qual lado da linha o novo dado se encontra. Nem sempre é possível separar linearmente os dados com um único hiperplano, e existem técnicas específicas para resolver esse caso, assim como para classificação multi-classes utiliza-se de uma área em contrapartida a uma linha.

Figura 7 – Classificação usando SVM



Fonte: Baseado em (GÉRON., 2017)

2.4.4 Transformers e LLMs

Graças aos transformers e sua contribuição especial para a criação dos Grandes Modelos de Língua (no inglês LLMs - Large Language Models) muitas das tarefas relacionadas a Inteligência Artificial foram repensadas e ganharam novos propósitos.

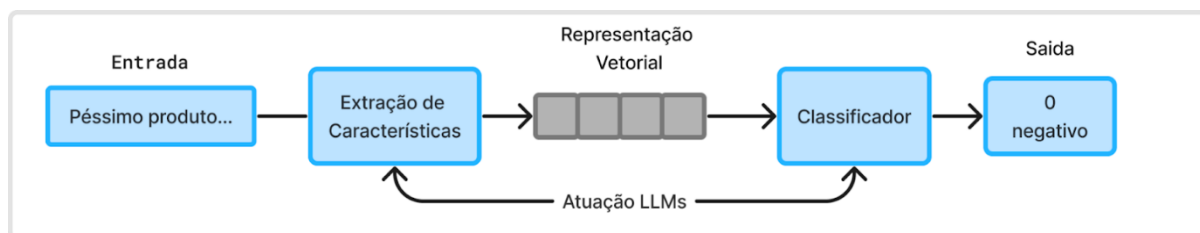
A tarefa de classificação de textos segue a mesma linha, sendo uma das mais comuns tarefas em processamento de língua natural. O impacto causado pelos LLMs na categorização é significativo e deve ser levado em consideração quando estamos projetando tarefas relacionadas.

Para afunilar um pouco essa vasta área, o foco será em modelos pré-treinados com uma grande quantidade de dados, que são usados para a categorização de texto. Esses modelos também dão a possibilidade do chamado ajuste fino, para trazer melhores resultados dependendo do projeto e contexto.

A etapa de partida escolhida foi a classificação supervisionada, visto que existem algumas outras opções para essa técnica, como as chamadas *few-shot* e *zero-shot*. Nesse tipo de classificação, como comentado para alguns exemplos anteriores que seguem a mesma ideia, durante a fase de treinamento, para que o modelo possa aprender, cada entrada tem já definida uma categoria alvo. Na Figura 8 ilustra uma representação simplificada do processo, e o ponto em questão é justamente que os LLMs podem ser utilizados para ambas as funcionalidades

discutidas anteriormente. O procedimento apresentado a seguir só se torna viável à medida que ele possa ser medido, pois uma classificação textual completamente aleatória não traria benefícios. Este procedimento será então utilizado em conjunto com uma avaliação do modelo classificador, que será apresentada na seção 2.5 Avaliação de Classificadores.

Figura 8 – Procedimento para classificação de texto supervisionada



Fonte: Baseado em (ALAMMAR et al., 2024)

Para escolher o modelo correto para a tarefa, é preciso levar em conta que cada um deles terá um desempenho considerando a seleção correta de parâmetros como linguagem da escrita, arquitetura do modelo, tamanho do corpus, velocidade de execução, dentre outros. As escolhas mais comuns são os conhecidos Generated Pretrained Transformers (família GPT), como o ChatGPT (OpenAI, 2024), que são excelentes em abrangência e generalização, ou os modelos BERT (Google, 2018), que são excelentes em passar pelo processo de ajuste fino e serem ótimos com tarefas específicas. No caso de tarefas como a deste trabalho, a escolha dos modelos BERT faz bastante sentido, visto que o objetivo é uma tarefa de contexto específico.

Afinal, qual a vantagem do BERT (*Bidirectional Encoder Representation from Transformer*) para modelos vistos até então? Bem, ele é considerado o estado da arte no que se diz respeito a modelos de *embedding*. Sua principal característica de destaque é ser um modelo que se baseia em contexto, diferentemente de outros como o Word2Vec que são livres de contexto (RAVICHANDIRAN, 2021). Na prática isso quer dizer que o BERT é capaz de gerar *embeddings* diferentes para a palavra “maga”, nas seguintes frases:

“Eu adoro suco de manga”

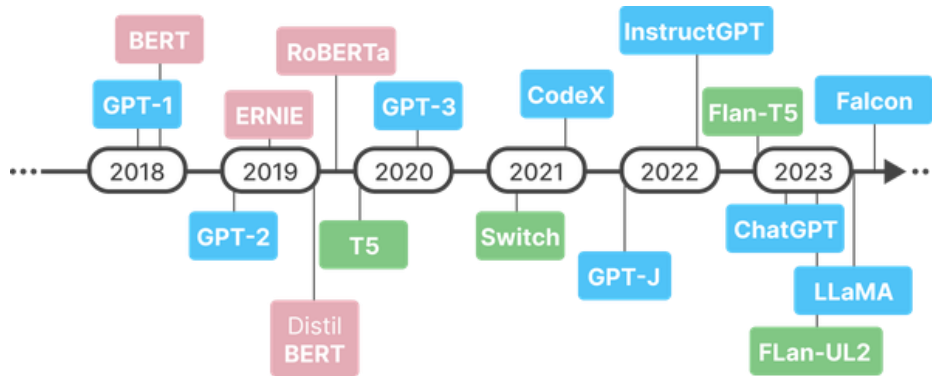
“Minha camiseta tem um furo na manga”

Para nós, dizer que as palavras têm significados diferentes nas frases é muito simples, mas para os modelos de Inteligência Artificial como Word2Vec, ambas teriam recebido o mesmo *embedding*.

A Figura 9 mostra uma linha do tempo da criação de alguns LLMs. Mesmo tendo feito a escolha inicial, ainda sim existem diversos tipos de variações que partem do BERT, que são

criadas pensando nos tipos de aplicação, como dados multilinguísticos, domínios específicos de treinamento, etc. que podem ser uma escolha apropriada.

Figura 9 – Linha do Tempo de lançamento de Grandes Modelos de Linguagem



Fonte: ALAMMAR et al. (2024)

2.4.5 SBERT - Sentence Transformers

Os modelos de linguagem baseados em transformers trouxeram uma nova perspectiva com relação ao desempenho em certos tipos de tarefas, dentre elas uma das importantes etapas apresentadas no pipeline proposto por esse trabalho, a extração de características. Apesar do BERT original ser excelente em tarefas de classificação, ele não é muito performático ao identificar similaridade de sentenças, pois requer que elas sejam concatenadas para então serem processadas, elevando o custo computacional de forma diretamente proporcional ao tamanho das sentenças.

Em 2019 foi apresentada uma modificação da rede BERT que tinha como objetivo encontrar estruturas semânticas similares, mas com um ganho significativo de performance se comparado aos modelos em que se baseia, dado a forma de estruturação e modo de operação, fazendo uso de redes do tipo *siamese and triplet* e processando as sentenças de forma separada. O SBERT ou Sentence-BERT busca trazer uma solução para criação dos *embeddings* com contexto semântico, tarefas essas que poderiam demorar de 65 horas usando o BERT/RoBERTa chegam a levar 5 segundos com o SBERT (REIMERS e GUREVYCH, 2019, p.1).

2.5 Avaliação de Classificadores

Uma vez treinado os classificadores com os dados que deseja que eles aprendam sobre o comportamento, de alguma forma, é necessário saber se o aprendizado foi efetivo e irá ser performático no cenário de utilização previsto, ou se é necessário voltar para a mesa de projetos e testar novas hipóteses e outros modelos. Para avaliar e comparar o desempenho, existem algumas métricas que são comumente utilizadas dentro do cenário de aprendizado de máquina, , que serão utilizadas ao longo deste trabalho.

Acurácia (Accuracy): É a medida de proporção entre as predições feitas corretamente pelo modelo e o total geral de predições.

$$Acurácia = \frac{\text{predições corretas}}{\text{predições totais}}$$

Matriz de Confusão: é uma tabela normalmente usada para descrever a performance do modelo de classificação, ela resume os valores preditos contra os valores reais na porção de teste.

Precisão (Precision): a proporção de verdadeiros positivos entre todas as instâncias classificadas como positivas.

$$sensibilidade = \frac{\text{verdadeiro positivo}}{\text{verdadeiro positivo} + \text{falso positivo}}$$

Sensibilidade (Recall): a proporção de verdadeiros positivos entre todas as instâncias que realmente são positivas.

$$sensibilidade = \frac{\text{verdadeiro positivo}}{\text{verdadeiro positivo} + \text{falso negativo}}$$

F1-Score: a média harmônica da precisão e recall. É útil quando há um desequilíbrio entre classes positivas e negativas.

$$F1\ score = \frac{\text{precisao} \times \text{sensibilidade}}{\text{precisao} + \text{sensibilidade}}$$

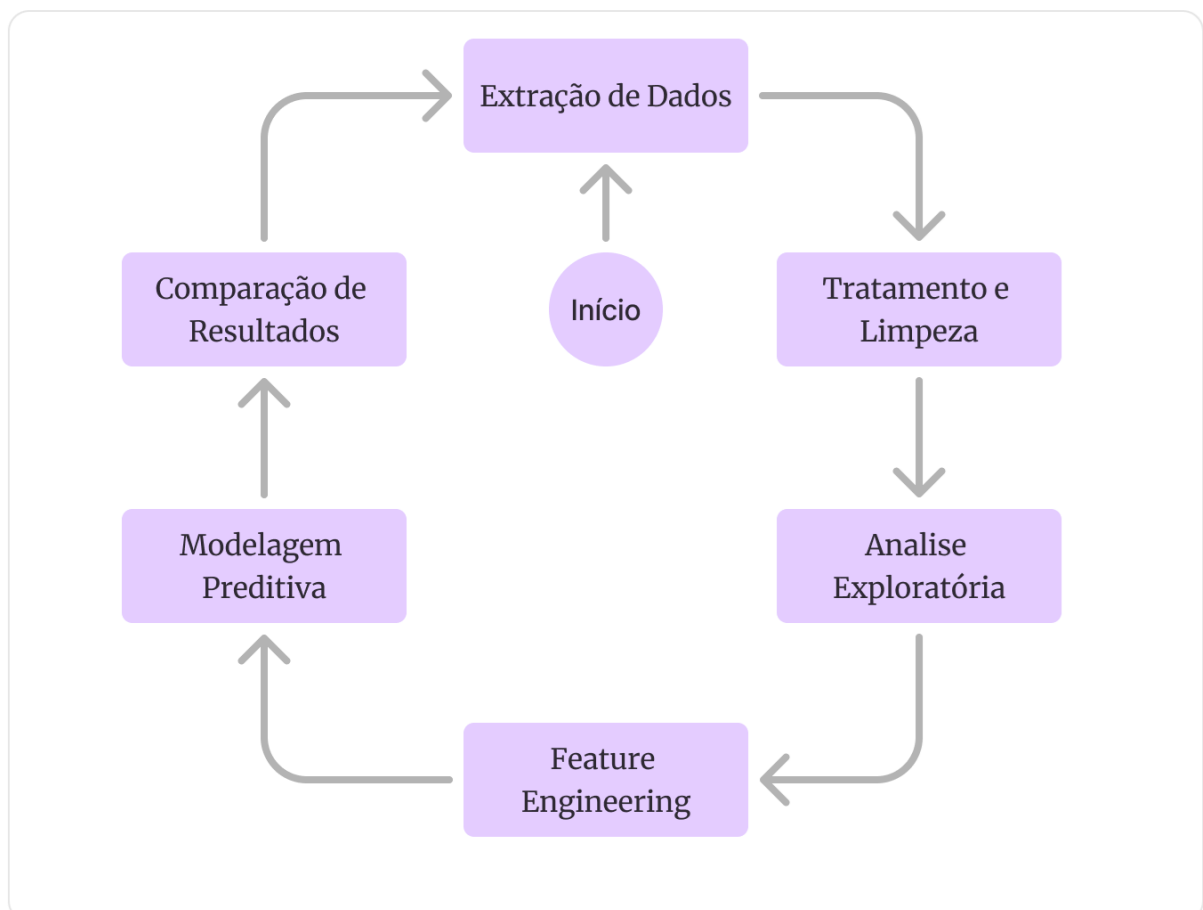
3 METODOLOGIA

Para o desenvolvimento da solução proposta, utilizou-se um conjunto de dados extraídos diretamente do software JIRA. Inicialmente, esses dados passarão por um tratamento preliminar para eliminar possíveis inconsistências decorrentes de falhas no preenchimento dos campos. Após essa etapa, o conjunto de dados será dividido em dois subconjuntos: treinamento e teste. Esses subconjuntos serão então submetidos aos classificadores para avaliação.

Ao longo deste trabalho, será comparado o desempenho de diversas técnicas de classificação. Particularmente, técnicas que consideram o contexto dos textos tendem a apresentar maior precisão, uma vez que os dados mais informativos são provenientes de textos livres. Esses textos contêm descrições detalhadas das tarefas de desenvolvimento, proporcionando um contexto rico para a classificação, mas até esse ponto essa é uma expectativa, sem embasamento estatístico.

A Figura 10 ilustra como funcionará o processo de trabalho seguido no estudo proposto, a seguir serão descritas as atividades e resultados obtidos dentro de cada uma destas etapas.

Figura 10 – Fluxo de trabalho proposto



3.1 Métricas de Avaliação

Este trabalho se encaixa, predominantemente, em um contexto de pesquisa quantitativa. A Pesquisa quantitativa se caracteriza pela coleta e análise de dados para extrair padrões e relações entre variáveis que podem ser generalizadas para outras populações. Esse é justamente o objetivo do trabalho, onde os dados serão submetidos a uma avaliação de desempenho de classificadores, com intuito de obter uma compreensão quantitativa e objetiva do impacto dessas técnicas na precisão das estimativas de tempo e esforço no desenvolvimento de software.

Os resultados obtidos a partir da aplicação dos classificadores serão analisados utilizando métricas como precisão, recall e F1-score, possibilitando uma avaliação robusta e detalhada do desempenho de cada técnica. Essa abordagem quantitativa fornecerá uma base sólida para conclusões sobre a eficácia das técnicas de classificação no contexto do desenvolvimento de software.

A escolha pelo tipo quantitativo se dá pela necessidade de analisar e quantificar o desempenho das técnicas utilizando dados extraídos de softwares de gestão de projetos, por meio de análise numérica e métricas de desempenho. Além disso, pode-se dizer que a escolha é motivada pelos seguintes fatores:

Objetividade: A análise de dados numéricos permite uma avaliação mais objetiva e precisa das técnicas de classificação.

Generalização: Os resultados obtidos podem ser generalizados para outras populações ou contextos semelhantes, aumentando a aplicabilidade dos achados.

Comparabilidade: A utilização de métricas quantitativas facilita a comparação direta entre diferentes abordagens de classificação, identificando quais métodos são mais eficazes.

Reprodutibilidade: Os procedimentos quantitativos são replicáveis, permitindo que outros pesquisadores validem e reproduzam os resultados.

3.2 Amostragem e Extração de Dados

No contexto do trabalho, visto que buscou-se extrair registros diretamente do software JIRA para o treinamento e avaliação, uma amostragem estratificada se enquadra muito bem, uma vez que os tipos de tarefa que são definidos na própria ferramenta, são características relevantes para a análise bem como para a divisão da população em grupos.

A coleta dos dados é feita diretamente no software, por meio da funcionalidade de exportação de resultados de pesquisa. Neste ponto, pode-se utilizar de diversos recursos da

própria ferramenta para filtrar os dados, evitando retrabalho na etapa de limpeza, importante também selecionar corretamente quais campos deverão fazer parte da exportação, visto que por padrão o JIRA não traz todos os campos que podem ser relevantes nas próximas etapas.

A escolha dos campos relevantes para extração vai depender muito do contexto de cada equipe e da organização dos gestores do JIRA, visto que mesmo os campos customizados podem ser de relevância para as análises, a Tabela 1 mostra uma lista de características escolhidas para esse trabalho, com algumas informações relacionadas ao *dataset* utilizado de exemplo, e logo a seguir uma breve explicação do campo e sua importância na escolha:

Tabela 1 – Distribuição de características

Característica	Tipo	Valores Únicos	Valores Nulos
Project	String	55	0
Key	String	5820	0
Summary	String	5676	0
Priority	String	9	0
Status	String	69	0
Issue Type	String	39	0
Story Points	Float	50	0
Description	String	4809	874

- **Projeto (*Project*):** A organização do software JIRA define que as tarefas estão diretamente ligadas a um projeto, e dentro da gestão da empresa podem existir diversos projetos acontecendo em paralelo. Na prática normalmente se atrela um projeto a um determinado produto, e isso pode ou não ser de extrema relevância para a análise. Se o intuito é fazer o treinamento baseado no contexto de apenas um produto, este campo deve ser utilizado na filtragem, caso o objetivo seja gerar um classificador que pode ser usado para qualquer projeto da empresa, deve-se considerar então usar dados de todos os projetos no treinamento.
- **Chave (*Key*):** Campo utilizado como identificador único de cada tarefa.
- **Resumo (*Summary*):** Campo destinado a um breve resumo da tarefa, pode conter informações relevantes dado o contexto de trabalho da equipe, pois costumam conter dados de sinalização importantes.

- **Prioridade (*Priority*):** Muito utilizado em alguns projetos e desconsiderado em outros, normalmente possui maior relevância para tarefas que são destinadas à resolução de problemas encontrados no software já funcional, como por exemplo bugs.
- **Estado (*Status*):** Utilizado para filtrar os dados, em alguns casos faz sentido considerar diversos tipos de estado. Algumas tarefas podem ter recebido seu campo story points antes de serem finalizadas, e dependendo do intuito do treinamento, essas tarefas podem ser consideradas. Em alguns casos deseja-se considerar apenas tarefas que já foram finalizadas. No escopo deste trabalho, não há necessidade de considerar apenas tarefas finalizadas.
- **Tipo de problema (*Issue Type*):** Muito utilizado para filtrar dados que empregam valor nos produtos, problemas identificados por clientes ou débitos técnicos.
- **Pontos de História (*Story Points*):** Campo que é definido como objetivo da classificação para este trabalho. Esse é um campo numérico que denota a complexidade de uma tarefa, normalmente está diretamente relacionado ao tempo que essa tarefa levará para ser feita, porém muito menos dependente do contexto da equipe. Por exemplo, um time acredita que a complexidade de uma tarefa é de 5 pontos, e leva 7 dias para finalizar essa tarefa. Já em outra equipe a tarefa discutida também é considerada de complexidade 5, porém para o contexto da segunda equipe essa leva apenas 2. Ambas são classificadas da mesma maneira, mas ao fazer uma correlação de pontuação com o tempo, levando em consideração apenas o contexto da equipe, pode-se adquirir valores mais precisos de estimativa temporal.
- **Descrição (*Description*):** Campo destinado à descrição dos problemas de negócio a serem resolvidos e requisitos a serem contemplados. É um campo extremamente contextual que pode agregar bastante na classificação, porém exige maior atenção no tratamento dos dados.

3.3 Tratamento e Análise

Uma vez feita a coleta inicial dos dados, deve-se considerar uma limpeza e preparação dos dados, mesmo tendo utilizado os filtros do software de gestão, pois alguns detalhes são mais bem tratados com ferramentas e técnicas próprias. Em seguida, esses dados serão submetidos aos classificadores, coletando os resultados e comparando as análises.

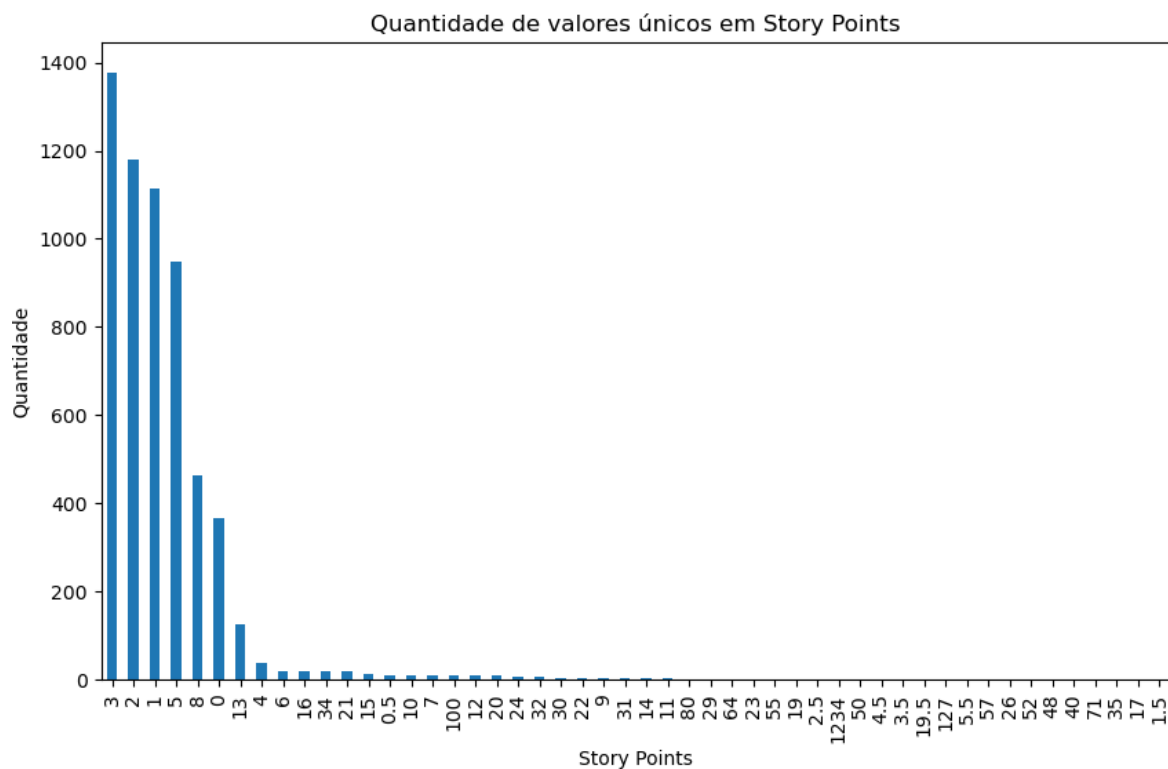
3.3.1 Exploração e pré-processamento

Após a extração inicial, uma exploração inicial dos dados pode ser feita com o intuito de visualizar melhor os dados, de forma mais ordenada. O objetivo é confirmar algumas percepções iniciais que se tem olhado para os dados de forma isolada, mas também conseguir

novas perspectivas. Para tal busca-se algumas visualizações específicas, muito ditadas pelo escopo do trabalho, algumas delas estão descritas em seguida, juntamente com as percepções extraídas.

Para traçar o gráfico apresentado Figura 11, deve-se primeiro converter o campo *story points* para o tipo número, porém para o *dataset* de exemplo a conversão resultava em alguns resultados NaN, indicando um erro na conversão. Esse erro pode ser causado devido a alguns motivos, e ao tentar resolvê-los dá-se início a etapa de pré-processamento. Um possível exemplo de problema é que a extração do JIRA pode trazer valores decimais separados por vírgula (ex. 0,5) e o código de conversão estar esperando por números separados por ponto (ex. 0.5), então é necessário tratar esse caso e talvez outros para garantir que na totalidade de entradas do *dataset*, nenhum valor do campo “*Story Points*” fique como nulo após a conversão.

Figura 11 – Gráfico exibindo quantidade de valores únicos no campo *Story Points*



O gráfico apresentado na Figura 11 mostra como uma porção dos dados se comporta com relação a variável objetivo. Apesar de uma dispersão razoável, existem alguns valores que, apesar de significativos para a análise, tem uma disparidade muito grande e ocorrência muito baixa em relação aos valores mais comuns. Esses valores mais recorrentes são os valores normalmente usados em estimativas de times utilizando metodologias de desenvolvimento

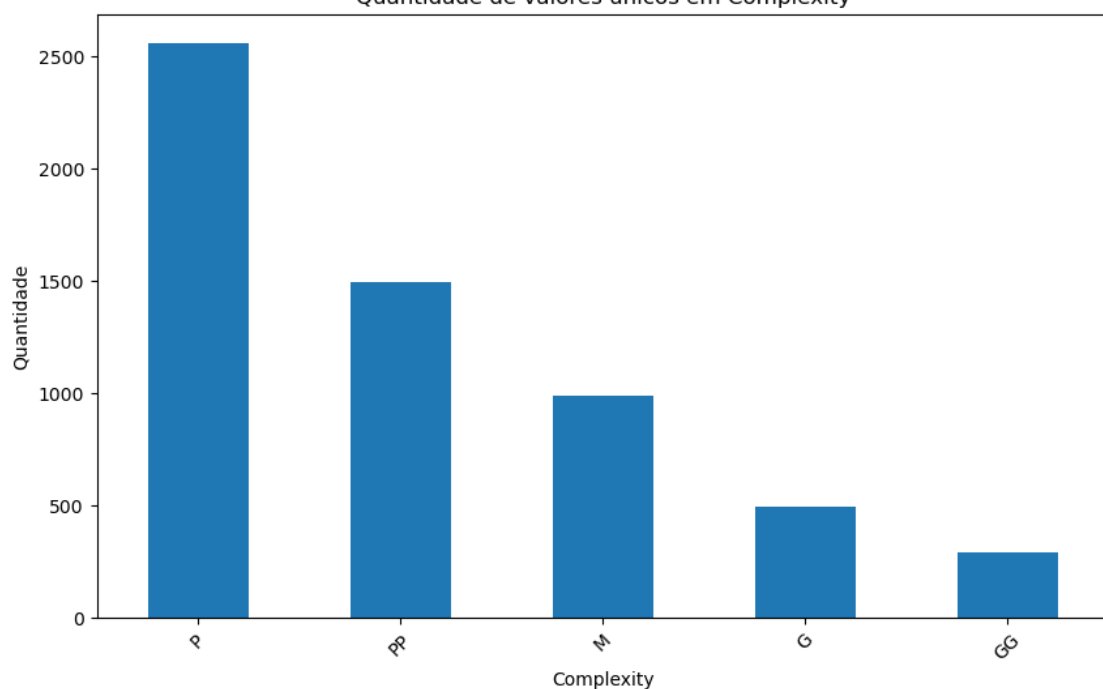
como Scrum, é muito comum encontrar times que utilizam uma regra para normalização de números de estimativa, como a conhecida sequência de Fibonacci.

Tendo isso em vista, essa análise inicial levanta uma possibilidade de normalização desses dados, algo que é, inclusive, praticado nos times no dia a dia em muitas situações. Uma possível regra para a normalização seria transformar os números no seu correspondente de Fibonacci mais próximo, ou mesmo transformar os números nos conhecidos *T-Shirt Size*, que identifica a complexidade das tarefas por tamanhos como de camisetas (PP, P, M, G...). Observe na Figura 12 a plotagem para a mesma base tratada da forma sugerida, usando os critérios dispostos na Tabela 2.

Tabela 2 – Critérios para tratamento do valor Story Points

Critério para o tratamento	Valor resultante
Story Points ≤ 1	PP
Story Points > 1 e ≤ 3	P
Story Points > 3 e ≤ 5	M
Story Points > 5 e ≤ 8	G
Story Points > 8	GG

Figura 12 – Gráfico exibindo quantidade de valores únicos no campo gerado
Quantidade de valores únicos em Complexity



No exemplo em questão, os valores foram mapeados de forma arbitrária pensando na relação direta dos números, mas no dia a dia de cada equipe é comum encontrar regras para determinar complexidade de tarefas que pode ser de grande valia na hora de definir os critérios para esse tratamento, levando em consideração o contexto de análise.

Ao analisar o gráfico apresentado na Figura 8, pode-se observar alguns pontos a respeito do comportamento dos dados,

- I. Existe uma maior quantidade de tarefas de complexidade baixa (PP e P)
- II. À medida que a complexidade aumenta as ocorrências diminuem. Em projetos de onde foram extraídos os dados exemplos, tem-se uma tendência a ter tarefas de complexidade menor, o que vai de encontro com uma busca constante na gestão de projetos ágeis, que é tentar trazer valor o mais rápido possível para os clientes.

Outra análise que se tem ao olhar para o gráfico da Figura 12, é que existe um desbalanceamento indesejado no *dataset* em questão, isso dificulta o treinamento dos algoritmos de aprendizado de máquina, deve-se então explorar algumas técnicas de pré-processamento para balancear esses dados.

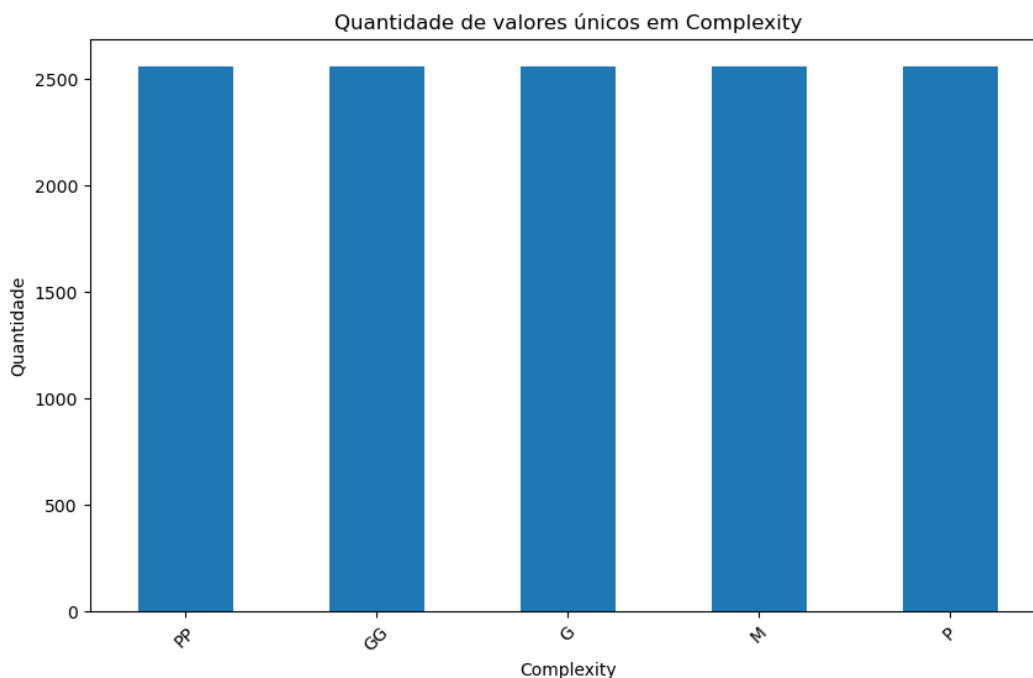
A primeira alternativa é tentar voltar para a extração e conseguir um novo *dataset* mais balanceado que, dependendo da quantidade de dados disponíveis, pode ser viável. Para o escopo deste estudo, serão tratadas algumas técnicas destinadas a resolver essa questão.

O desbalanceamento de classes é um problema já conhecido e muito estudado ao longo dos últimos anos, em um de seus trabalhos, Japkowicz (2000) aponta que, quanto maior a complexidade do problema em questão, maior a tendência de os resultados serem afetados pelo desbalanceamento, ao passo que problemas simples como de classes binária são menos suscetíveis a essa característica.

Por outro lado, alguns autores defendem que, dado o avanço das capacidades de aprendizado dos modelos atuais, não se deve tentar resolver o problema do desbalanceamento, já que esse problema representa uma característica verdadeiramente presente no cenário real. Cabe aos modelos aprenderem a lidar com tal questão (Huyen, 2022).

Como o intuito desse trabalho é avaliar performance dos classificadores, não faria sentido buscar a perfeição na otimização dos dados em uma etapa inicial. Utilizar técnicas simples como amostragem aleatória (*random oversampling*) pode ser suficiente para aumentar significativamente os resultados obtidos, eventualmente ao longo do trabalho pode ser necessário fazer alguns ajustes e utilizar de outras técnicas de balanceamento para obter melhores resultados. O gráfico apresentado na Figura 13 mostra o resultado com relação a quantidade de dados após a aplicação da técnica sugerida.

Figura 13 – Quantidade de valores únicos no campo *Complexity* após *random oversampling*



O *dataset* agora possui a mesma quantidade de dados para todas as classes, mas vale lembrar que os dados são simplesmente copiados para alcançar o equilíbrio graças à técnica de *random oversampling*, o que pode gerar *overfitting*. Após a etapa de treinamento e avaliação dos resultados, pode ser necessário retornar para o balanceamento e utilizar técnicas diferentes para minimizar erros, isso será discutido em etapas posteriores deste estudo.

Com o balanceamento inicial feito, pode-se começar a pensar em fazer uma predição inicial, usando algum algoritmo de implementação mais simples, para que sirva de parâmetro conforme os ajustes de pré-processamento forem feitos, dando continuidade no fluxo de trabalho descrito anteriormente na Figura 6. Para isso, uma seleção pequena de características poderia ser inicialmente usada, e em seguida ir aumentando o número de características e comparando os resultados. No início do trabalho foi levantada a hipótese da importância dos campos descritivos como *Summary* e *Description*, mas até este momento, isso ainda não foi testado e comprovado, portanto faz sentido iniciar os testes sem essas características, e avaliar a melhora nos resultados ao acrescentá-las ao pipeline.

Na Tabela 1 foram apresentados os campos iniciais do *dataset*, o trabalho com o campo objetivo já foi iniciado, o foco agora será em 3 outros campos, que também passaram por um pré-processamento antes de serem submetidos ao teste inicial, são eles “Project”, “Issue Type” e “Priority”, essas características, assim como o campo “Complexidade” gerado anteriormente, são consideradas categóricas, o que significa que recebem valores a partir de uma lista limitada

e fixa de possíveis valores, se comparando aos *enums* na ciência da computação (ABHISHEK; ABDELAZIZ, 2023). Diferente de um campo como o “Description” que possui praticamente apenas valores únicos, esses campos têm uma quantidade explícita de valores, e para facilitar o aprendizado de máquina, será feito a conversão desses campos para valores numéricos.

Ainda antes de aplicar as técnicas de *encoding* para esses campos, pode ser útil fazer uma exploração inicial em seus valores atuais em busca de inconsistências ou possíveis ajustes relacionados ao contexto do *dataset*. A característica *Issue Type*, por exemplo, é comumente utilizada pelas empresas, mas se esta permite que diversas pessoas façam a gestão desse campo no espectro organizacional, muitos nomes similares e inconsistências podem surgir, um tratamento foi feito para esse campo seguindo a mesma lógica da complexidade, fazendo a união de alguns valores, como por exemplo os valores “Support Request”, “PR - Support Request” e “[System] Service request”, que serão transformados todos em um mesmo valor chamado “Support Request”.

Após uma análise nos três campos citados, constatou-se que a redução de valores únicos foi viável apenas para um deles, o campo “Issue Type”, já para as características “Project” e “Priority”, esse tratamento não fez sentido, pois diferente do anterior onde 2 valores significam na prática a mesma coisa, para estes outros dois eles mantiveram sua importância singular dentro do escopo de negócio analisado. A Tabela 3 mostra os novos valores obtidos.

Tabela 3 – Resultado do tratamento de características

Característica	Valores únicos originalmente	Valores únicos após tratamento
Project	55	0
Priority	9	9
Issue Type	39	28

O próximo passo é o anteriormente mencionado *encoding*, e nesta etapa será feito o mesmo processo para as três características mencionadas acima, bem como para o campo de complexidade gerado anteriormente. A técnica utilizada será o *label encoding*, amplamente utilizado e disponibilizado em diversas bibliotecas do uso da ciência de dados, mas o que não impede a implementação customizada por se tratar de uma técnica relativamente simples, seu objetivo é transformar valores únicos em representações numéricas. Na Tabela 4 é possível observar o resultado dessa técnica para o campo “Complexity”, que foi gerado anteriormente:

Tabela 4 – *Encoding* do campo Complexidade

Valor Original	Valor após <i>encoding</i>
PP	4
P	3
M	2
G	0
GG	1

Uma vez recebido as novas características, o *dataset* ganha 4 novas colunas, representadas por novos nomes escolhidos arbitrariamente, mas sua ligação é direta com as colunas das quais eles foram gerados. Na etapa de treinamento descrita na seção seguinte, apenas esses campos serão utilizados, e posteriormente serão acrescentadas novas características do *dataset* original em busca de melhorar o modelo e seus resultados. A Figura 14 traz uma representação gráfica dos dados após esse tratamento.

Figura 14 – Representação gráfica do dataset após tratamento inicial

	Complexity_Encoded	Priority_Encoded	Issue_Type_Encoded	Project_Encoded
0	4	6	24	21
1	4	4	14	38
2	4	6	24	21
3	4	6	24	10
4	4	6	24	10
...	...	...	...	...
12795	4	4	23	19
12796	4	6	24	42
12797	4	6	24	22
12798	4	6	24	10
12799	4	6	6	37

3.3.2 Treinamento - Classificação/Predição usando algoritmos de AM

Com os dados tratados, pode-se dar então início a etapa de treinamento dos modelos. O treinamento consiste em utilizar dos dados em que já se possui a resposta em relação a classe correta que estes pertencem e “esconder” essa resposta do algoritmo, para que a máquina tente chegar nas respostas por conta própria. As primeiras etapas desse processo são muito parecidas independente da implementação do modelo, dentre elas está a identificação de quais serão as características (features) e qual o objetivo de classificação (target), que é justamente o que se deseja que o algoritmo faça a predição.

Após identificar features e target, separa-se então os dados em conjuntos de treinamento e de teste. A porção de treinamento é utilizada para aprendizado, o algoritmo utilizará essa porção para aprender sobre o comportamento dos dados, nessa porção os dados da variável *target* precisam estar disponíveis para o algoritmo para que ele aprenda o comportamento. Já na porção de teste, que será uma porção dos dados onde o target será “escondido”, ficará a cargo do algoritmo utilizar dos padrões aprendidos e, com o modelo aprendido e usando apenas das características, chegar a uma conclusão de qual será o target. Com isso é possível então executar os algoritmos e, ao comparar o valor alcançado nas variáveis target da porção de teste com os dados originais pode-se observar algumas métricas de precisão, para chegar a uma conclusão do quão eficiente foi o algoritmo naquele *dataset*.

Com os dados tratados, pode-se então dar início à etapa de treinamento dos modelos. O treinamento consiste em utilizar dados nos quais já se conhece a resposta correta em relação à classe a que pertencem e "esconder" essa resposta do algoritmo, para que ele tente chegar às respostas por conta própria. As primeiras etapas desse processo são muito parecidas, independentemente da implementação do modelo. Entre elas, estão a identificação de quais serão as características (features) e qual o objetivo de classificação (target), que é justamente o que se deseja que o algoritmo realize a predição.

Uma vez identificadas as features e o target, os dados são então separados em conjuntos de treinamento e de teste. O conjunto de treinamento é utilizado para calibrar o algoritmo, que o utilizará para aprender sobre o comportamento dos dados. Nesse estágio, o algoritmo tem acesso ao target e às features em conjunto. Já no conjunto de teste, que será uma parte dos dados onde o target será "escondido", caberá ao algoritmo utilizar os padrões aprendidos no conjunto de treinamento para, por meio apenas das características, chegar a uma conclusão sobre qual será o target.

Dessa forma, é possível executar os algoritmos e, ao comparar o valor alcançado nas variáveis target do conjunto de teste com os dados originais, obter algumas métricas de precisão. Essas métricas permitirão avaliar a eficiência do algoritmo naquele conjunto de dados.

3.3.2.1 Treinamento com *Dataset* Simplificado

Utilizando os dados descritos na Figura 14, junto com os conceitos sobre treinamento citados anteriormente, é possível finalmente testar a capacidade de predição de alguns algoritmos, para isso foram utilizadas bibliotecas já preparadas para esse fim como a *sklearn*, foram escolhidos 3 algoritmos inicialmente, *Logistic Regression*, *Random Forest* e *GD Boosting*. A Tabela 5 mostra um comparativo dos resultados de cada um deles, usando métricas comuns para avaliação.

Com relação aos parâmetros utilizados, para a regressão logística, foram usadas as configurações de um número máximo de iterações de 10000, e o parâmetro “multi_class” definido com o valor ‘multinomial’, usando sempre a relação treinamento e teste de 70/30. Para os testes com *Random Forest*, a relação treinamento e teste também foi 70/30 e o número de estimadores foi definido como 100. Para *GDBosting*, mesma relação de treinamento e teste e o número máximo de iterações foi definido como 100. Todas as métricas de avaliação utilizadas (*precision*, *recall* e *f1 score*) levaram o parâmetro de *average* com o valor ‘weighted’. Essa seleção de parâmetros se manteve como regra para todas as avaliações apresentadas nesse trabalho.

Tabela 5 – Comparativo de resultado no *dataset* simplificado

Método	Relação Treinamento: Teste	<i>Accuracy</i>	<i>Precision</i>	<i>Recall</i>	<i>F1 Score</i>
<i>Logistic Regression</i>	70:30	0.2388	0.2300	0.2388	0.2043
<i>Random Forest</i>	70:30	0.5221	0.5208	0.5221	0.5159
<i>Gradient Boosting</i>	70:30	0.5171	0.5175	0.5171	0.5136

Uma vez testados os algoritmos, pode-se comparar os valores e chegar à conclusão de que, para essa execução, a *Random Forest* obteve melhores resultados, porém o *Gradient Boosting* não teve um resultado muito pior, como foi o caso da regressão logística. O objetivo

agora é adicionar os campos de texto livre (*Description* e *Summary*) fazendo uso de algumas técnicas específicas para transformá-los em números, e com isso tentar chegar em resultados melhores dos que os apresentados na tabela 5.

3.3.2.2 Treinamento com *Dataset* Simplificado e campos descritivos

O próximo passo foi trabalhar o campo *Summary*, que possui uma breve descrição textual do objetivo de cada tarefa, e por conta dessa característica, é preciso recorrer a alguma forma de transformar esse dado em uma representação numérica para facilitar o trabalho dos classificadores. Na sessão de 2.3 Extração de Características, foi feita uma breve descrição a respeito de técnicas comuns para executar essa tarefa e, para chegar nos valores apresentados na Tabela 6 o SBERT foi escolhido para gerar os *embeddings*.

A principal decisão de configuração ao utilizar o SBERT é de qual modelo pré-treinado ele irá utilizar, existem muitas escolhas comuns, entre uma vasta lista de possibilidades disponível, por exemplo, no site *Hugging Face*, mas uma escolha compacta e simples, que foi escolhida para esse trabalho é o "all-MiniLM-L6-v2".

Uma vez utilizado, o SBERT ao aplicado para esse fim nos valores de *Summary* gerou uma quantidade significativa de novas variáveis para serem analisadas, pensando no conceito matricial, a representação se tratava de uma matriz de 12800 linhas e 387 colunas. Gerando os *embeddings*, a coluna *Summary* então é representada por novas 384 colunas (depende do modelo escolhido), que foram arbitrariamente nomeadas de “*summary_embedding*”, para treinar e executar cada um dos modelos de classificação e, em seguida, poder comparar os resultados, fez-se uma união das colunas geradas com o *dataset* simplificado, resultado em algo similar ao representado na Figura 15:

Figura 15 – Representação gráfica do *dataset* após união com *embedding*

	Priority_Encoded	Issue_Type_Encoded	Project_Encoded	summary_embedding_0	summary_embedding_1	summary_embedding_2	...	summary_embedding_382	summary_embedding_383
0	6	24	21	-0.160203	0.013820	0.005409	...	0.047071	0.022134
1	4	14	38	-0.037549	0.078765	-0.003598	...	0.027787	0.002177
2	6	24	21	-0.035217	0.011125	-0.069525	...	-0.049847	-0.003025
3	6	24	10	-0.013494	0.013726	-0.089642	...	0.059024	-0.039124
4	6	24	10	-0.074191	-0.018799	-0.041839	...	0.084990	-0.019561

5 rows × 387 columns

Os resultados do experimento de classificação com esse novo *dataset* é mostrado na Tabela 6.

Tabela 6 – Comparativo de resultado no dataset simplificado + summary

Método	Relação Treinamento: Teste	<i>Accuracy</i>	<i>Precision</i>	<i>Recall</i>	<i>F1 Score</i>
<i>Logistic Regression</i>	70:30	0.4604	0.4520	0.4604	0.4532
<i>Random Forest</i>	70:30	0.8718	0.8766	0.8718	0.8735
<i>Gradient Boosting</i>	70:30	0.8638	0.8625	0.8638	0.8630

A mesma ideia será aplicada ao segundo campo descritivo que o *dataset* possui, o campo “*description*”. Esse campo, de forma geral, tende a ter uma quantidade muito maior de caracteres, além de que o software JIRA possibilita a inserção de imagens e formações especiais para corpo a descrição de tarefas, o objetivo geral é descrever da melhor forma possível o trabalho que precisa ser feito para dar ao desenvolver, ou a quem for executar a tarefa, então é comum que este campo tome grandes proporções.

Para este trabalho, não foram feitos tratamentos específicos das imagens contidas, porém optou-se por manter caracteres especiais de marcação (que indicam negrito, por exemplo) no valor final a ser criado os *embeddings*. Outro detalhe importante identificado no *dataset* exemplo foi que se identificou que algumas descrições estavam nulas, ou identificadas como NaN no pandas. Na escolha entre descartar essas entradas e mantê-las, optou-se pela segunda opção, pois acredita-se que mesmo o valor vazio pode ter relação com o tamanho da tarefa, como por exemplo tarefas muito simples, que não precisam de descrição detalhada, onde apenas o título poderia ser suficiente. Para tal, é importante transformar os campos em string vazias, pois o SBERT não é capaz de gerar *embedding* para NaN sem tratamento.

A Tabela 7 mostra o resultado obtido treinando os classificadores utilizando apenas o *dataset* simplificado e o campo *description*, excluindo do treinamento o campo *Summary* comentado anteriormente, já a Tabela 8 mostra os resultados utilizando ambos os campos, onde se imaginava que seriam obtidos os melhores resultados. A Tabela 9, Tabela 10 e Tabela 11 apresentam os resultados utilizando apenas os campos descritivos de forma isolada.

Tabela 7 – Comparativo de resultado no *dataset* simplificado + *description*

Método	Relação Treinamento: Teste	<i>Accuracy</i>	<i>Precision</i>	<i>Recall</i>	<i>F1 Score</i>
<i>Logistic Regression</i>	70:30	0.4544	0.4471	0.4544	0.4430
<i>Random Forest</i>	70:30	0.8281	0.8307	0.8281	0.8284
<i>Gradient Boosting</i>	70:30	0.8195	0.8207	0.8195	0.8198

Tabela 8 – Resultados no *dataset* simplificado + *summary* + *description*

Método	Relação Treinamento: Teste	<i>Accuracy</i>	<i>Precision</i>	<i>Recall</i>	<i>F1 Score</i>
<i>Logistic Regression</i>	70:30	0.5424	0.5324	0.5424	0.5342
<i>Random Forest</i>	70:30	0.8742	0.8808	0.8742	0.8760
<i>Gradient Boosting</i>	70:30	0.8693	0.8691	0.8693	0.8692

Tabela 9 – Resultados no *dataset* com o campo *summary* isolado

Método	Relação Treinamento: Teste	<i>Accuracy</i>	<i>Precision</i>	<i>Recall</i>	<i>F1 Score</i>
<i>Logistic Regression</i>	70:30	0.4362	0.4266	0.4362	0.4289
<i>Random Forest</i>	70:30	0.8565	0.8628	0.8565	0.8583
<i>Gradient Boosting</i>	70:30	0.8565	0.8569	0.8565	0.8565

Tabela 10 – Resultados no *dataset* com o campo *description* isolado

Método	Relação Treinamento: Teste	<i>Accuracy</i>	<i>Precision</i>	<i>Recall</i>	<i>F1 Score</i>
<i>Logistic Regression</i>	70:30	0.4451	0.4409	0.4451	0.4312
<i>Random Forest</i>	70:30	0.7690	0.7939	0.7690	0.7690
<i>Gradient Boosting</i>	70:30	0.7685	0.7893	0.7685	0.7672

Tabela 11 – Resultados no *dataset* com o campo *description* + campo *summary*

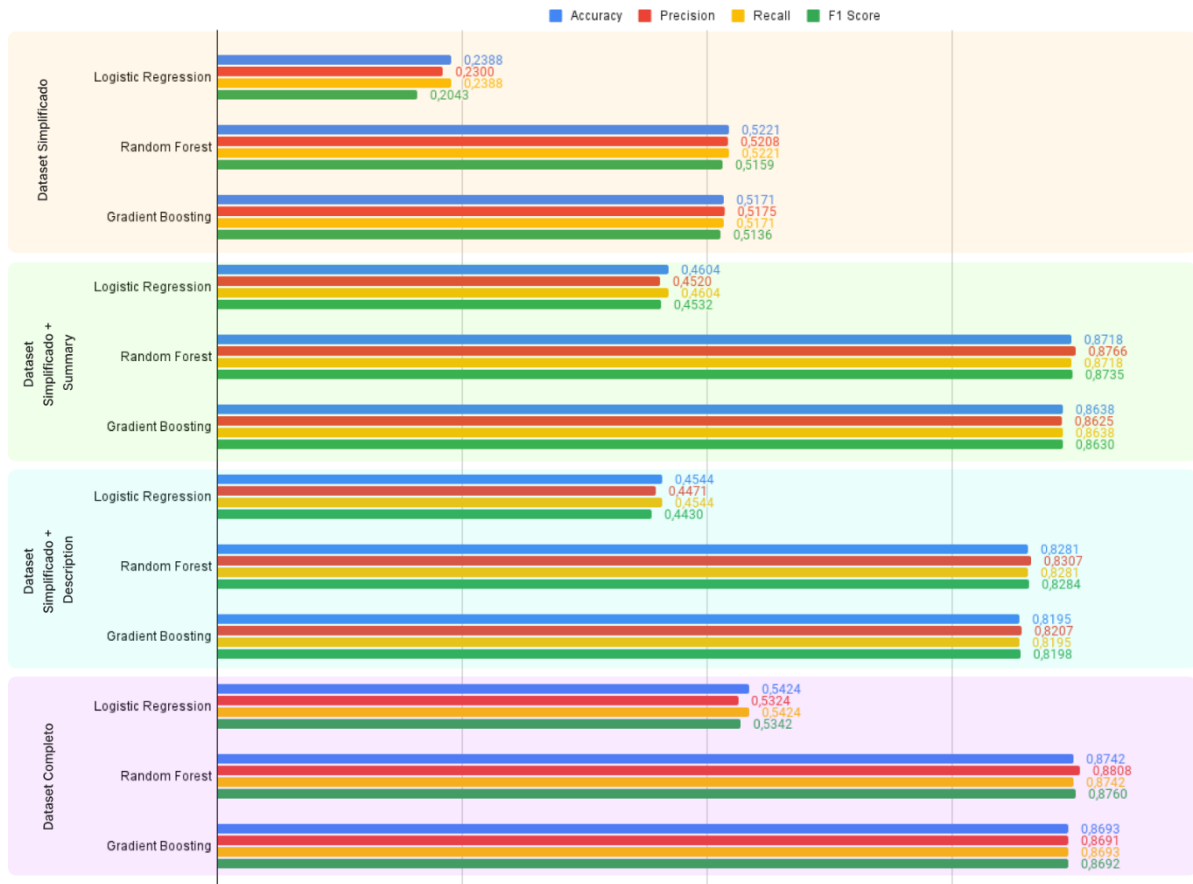
Método	Relação Treinamento: Teste	<i>Accuracy</i>	<i>Precision</i>	<i>Recall</i>	<i>F1 Score</i>
<i>Logistic Regression</i>	70:30	0.5396	0.5285	0.5396	0.5313
<i>Random Forest</i>	70:30	0.8633	0.8680	0.8633	0.8647
<i>Gradient Boosting</i>	70:30	0.8607	0.8602	0.8607	0.8604

É importante lembrar que esses são os valores obtidos com um *dataset* construído nesse trabalho de conclusão de curso, com dados extraídos do JIRA utilizando por uma empresa em especial, mas imagina-se que mesmo dentro de realidades diferentes, a importância de cada campo mediante a uma organização e padronização mínima acabam por representar resultados de características similares, podendo variar para valores mais precisos ou menos, dado a qualidade e quantidade de dados disponíveis.

3.4 Discussão dos Resultados

Com os resultados obtidos para cada método em suas respectivas configurações, torna-se possível realizar uma comparação detalhada para avaliar a viabilidade e utilidade da aplicação dentro das expectativas de uso. Os valores resultantes são apresentados lado a lado, permitindo a indicação do modelo que melhor se adequa à classificação proposta neste estudo. Essa análise comparativa pode ser visualizada na Figura 16.

Figura 16 – Comparativo dos resultados obtidos



Ao analisar os dados apresentados, é possível observar que o melhor resultado obtido é proveniente do *dataset* completo, aquele que se possui a maior quantidade de informações relacionadas às tarefas. Essa era uma suposição inicial, que fica comprovada para o escopo deste trabalho, além dessa, outras observações podem ser extraídas, tal como o aumento significativo da precisão ao incluir o primeiro campo de texto livre “*Summary*”, inicialmente se imaginava que esse campo seria de grande valia para os resultados, mas não que o valor seria tão significativo, visto que apesar de um campo livre ele é mais objetivo e não costuma ter tantas informações, mas os resultados provam que mesmo essa descrição simples nesse escopo pode trazer mesmo que sozinha já ótimos resultados.

Os resultados obtidos com o campo “*Summary*”, em conjunto com o *dataset* inicial ou isoladamente, também demonstram uma perspectiva promissora para a aplicação prática dessas predições. Uma vez que, para criar tarefas, o responsável por tal atividade segue um fluxo de trabalho em que se pensa inicialmente nas ideias e o que precisa ser feito, em seguida transforma-se essas ideias em uma ou mais tarefas no JIRA. Muitas vezes elas são inicialmente definidas justamente com o campo “*Summary*”, ou seja, elas nascem já com esse campo, sendo

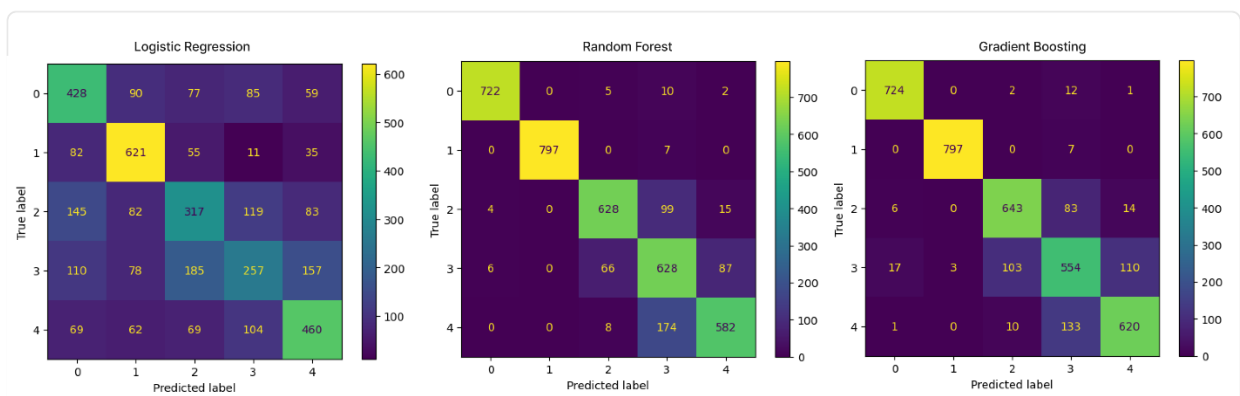
assim uma predição de complexidade com a ajuda da IA poderia acontecer logo na criação e, como os dados mostram, com uma precisão razoável.

Uma preocupação que surgiu ao longo do trabalho é a necessidade de gerar dados devido ao desbalanceamento do *dataset*, uma vez que um problema muito comum nesses casos é ocorrer overfitting, fazendo com que o treinamento se adeque demasiadamente aos dados pela quantidade de repetições. Nesse estudo olhando para a similaridade dos valores de *accuracy*, *recall* e *F1-Score* os testes e os modelos utilizados demonstram generalizar bem.

Para melhor interpretação das matrizes apresentadas na Figura 17, vale destacar que o *dataset* é o mesmo utilizado para obter os resultados da Tabela 8, e que possui os campos (após tratamentos como encoding e embeddings) *Priority*, *Issue Type*, *Project*, *Summary* e *Description*. Além disso a Tabela 4 mostra uma informação relevante de relação entre a Complexidade, que era a característica e seu valor após *encoding*.

Com isso pode-se notar que os erros, ao menos nos testes de *Random Forest* e *GDBosting*, tendem a ser para a complexidade mais próxima, por exemplo, onde o valor correto era 3 (Complexidade P) a maioria dos erros de predição foram em apontar como 4 (Complexidade PP) seguido de 2 (Complexidade M), em questões práticas esse erro é mais bem aceito do que o caso de prever algo que na verdade é um 4 (Complexidade PP) como 1 (Complexidade GG). Com esse dado, além do valor considerado bom de precisão, tem-se ainda uma boa perspectiva com relação aos possíveis erros, que não seriam tão impactantes na aplicação.

Figura 17 – Matriz de Confusão para os testes de dataset tratado



4 CONCLUSÕES E TRABALHOS FUTUROS

Esta seção tem como objetivo apresentar uma análise dos processos de extração e tratamento dos dados, bem como comentar os resultados obtidos. Durante a etapa de extração, utilizou-se a própria ferramenta de exportação oferecida pelo software JIRA para a obtenção dos dados iniciais, a ferramenta oferece suporte a pesquisas elaboradas que são executadas por meio de uma linguagem específica chamada JQL (*JIRA Query Language*). A utilização e refinamento da busca feita no JIRA garante uma pré-filtragem dos dados descritos nas sessões anteriores, como por exemplo a exclusão de projetos específicos ou garantindo valores não nulos.

Muito do conhecimento específico pode ser empregado nessa etapa, especialmente o aquele a respeito dos processos da empresa e a forma de gestão entre diferentes equipes, pois é possível que mesmo em produtos e contextos um pouco diferentes, quando questões arquiteturais e a forma de trabalho não são tão divergentes os dados podem ser utilizados em conjunto aumentando a representatividade.

Feita a extração inicial, foram necessárias algumas análises e tratamentos já com o pensamento voltado propriamente aos valores obtidos. Nem todos os campos precisavam ser tratados como campos de texto livre diretamente, como foi o caso da variável objetivo “*Story Points*”, que sofreu algumas alterações no formato ao longo do trabalho. Os campos de texto livre como “*Description*” e “*Summary*” foram submetidos ao SBERT para criar os *embeddings* de forma eficiente e significativa, levando em consideração similaridade semântica entre as sentenças.

Os modelos examinados incluem *Logistic Regression*, *Random Forest* e *Gradient Boosting*, e as métricas de *Accuracy*, *Precision*, *Recall* e *F1 Score* foram utilizadas para medir o desempenho de cada um em relação ao *dataset* extraído.

De forma geral, para o contexto do trabalho, olhando para a distribuição de classes ter sido trabalhada e o fato de que, o impacto gerado por uma predição ser feita pela complexidade diretamente adjacente, como pode-se constatar em uma das matrizes de confusão apresentada na Figura 17 acredita-se que a acurácia acima de 85% é considerada um bom resultado.

Para a utilização do conceito avaliado neste trabalho no dia a dia de equipes de desenvolvimento, existem algumas abordagens, como adicionar um passo manual no fluxo de trabalho como mencionado anteriormente, em que ao criar a tarefa esta é submetida a classificação e o valor obtido pode ser utilizado em estimativas e mesmo para entender se a tarefa está de um tamanho adequado para as necessidades da equipe ou mesmo dentro de

padrões esperados. Uma alternativa seria automatizar essa predição por meio do uso de funções gancho específicas do JIRA e, pelos mesmos motivos da alternativa manual, garantir que não haja necessidade de intervenção manual para obter esse valor. Ambas as formas se mantêm como campo aberto para análise e ficaram fora do escopo deste trabalho.

Para trabalhos futuros, seguindo o aqui apresentado, existe a possibilidade de um teste mais completo, utilizando modelos mais recentes de LLM e classificados, como o SVM e as próprias redes neurais pode melhorar ainda mais esse resultado.

Um tópico relacionado, mas que não foi abordado neste trabalho, é a definição do esforço despendido. Embora este estudo se concentre em estimativas e previsões de novos cenários, utilizar os dados já existentes para classificar e metrificar o tempo gasto em tarefas recém-concluídas também pode ser extremamente útil para a gestão de projetos. Isso é particularmente relevante quando se busca maior clareza na divisão de investimentos e na avaliação da performance das equipes. Um exemplo prático é a tentativa de determinar quanto do orçamento de uma equipe foi destinado ao desenvolvimento de novas funcionalidades em comparação ao retrabalho, débitos técnicos ou manutenção do produto atual. Como cada uma dessas atividades envolve diversos profissionais em diferentes momentos, o gestor que tenta mensurar o esforço total ao final do processo enfrenta inúmeros desafios.

REFERÊNCIAS

- FORBES. Top 10 biggest companies in the world by market cap in 2024. 2024. Disponível em: <https://www.forbesindia.com/article/explainers/top-10-largest-companies-world-market-cap/86341/1>. Acesso em: 15 jan. 2024.’
- MENON, V. M.; RAHULNATH, H. A. A novel approach to evaluate and rank candidates in a recruitment process by estimating emotional intelligence through social media data. In: 2016 INTERNATIONAL CONFERENCE ON NEXT GENERATION INTELLIGENT SYSTEMS, ICNGIS 2016, 2017. DOI: 10.1109/ICNGIS.2016.7854061.
- HU, X.; XIA, B.; SKITMORE, M.; CHEN, Q. The application of case-based reasoning in construction management research: An overview. *Automation in Construction*, v. 72, p. 65–74, 2016.
- KO, C.-H.; CHENG, M.-Y. Dynamic prediction of project success using artificial intelligence. *Journal of Construction Engineering and Management*, v. 133, n. 4, p. 316–324, 2007. DOI: 10.1061/(ASCE)0733-9364(2007)133:4(316).
- JIN, L.; ZHANG, C.; SHAO, X.; TIAN, G. Mathematical modeling and a memetic algorithm for the integration of process planning and scheduling considering uncertain processing times. *Proceedings of the Institution of Mechanical Engineers, Part B: Journal of Engineering Manufacture*, v. 230, n. 7, p. 1272–1283, 2016. DOI: 10.1177/0954405415625916.
- DAM, Hoa Khanh; et al. Towards effective AI-powered agile project management. In: 2019 IEEE/ACM 41ST INTERNATIONAL CONFERENCE ON SOFTWARE ENGINEERING: NEW IDEAS AND EMERGING RESULTS (ICSE-NIER), 2019. p. 41-44.
- TUNSTALL, Lewis; VON WERRA, Leandro; WOLF, Thomas. *Natural language processing with transformers*. O'Reilly Media, Inc., 2022.
- IDERHOFF, Nicolas. Alphabetical list of free/public domain datasets with text data for use in Natural Language Processing (NLP). 2024. Disponível em: <https://github.com/niderhoff/nlp-datasets>. Acesso em: 03 abr. 2024.
- GOOGLE. Dataset Search by Google. 2024. Disponível em: <https://datasetsearch.research.google.com/>. Acesso em: 03 abr. 2024.
- JAPKOWICZ, Nathalie. The class imbalance problem: significance and strategies. *Proceedings of the 2000 International Conference on Artificial Intelligence ICAI*, 2000.
- VAJJALA, Sowmya; MAJUMDER, Bodhisattwa; GUPTA, Anuj; SURANA, Harshit. *Practical natural language processing*. O'Reilly Media, Inc., 2020.
- PALMER, David D. Tokenisation and sentence segmentation. *Handbook of natural language processing*, p. 11-35, 2000.

KOCHMAR, Ekaterina. Getting started with natural language processing. Manning Publications, 2022.

HASTIE, Trevor; TIBSHIRANI, Robert; FRIEDMAN, Jerome. The Elements of Statistical Learning: Data Mining, Inference, and Prediction. 2. ed. New York: Springer, 2009.

WEBB, Geoffrey I.; KEOGH, Eamonn; MIIKKULAINEN, Risto. Naïve Bayes. Encyclopedia of machine learning, v. 15, n. 1, p. 713-714, 2010.

GÉRON, Aurélien. Hands-on machine learning with scikit-learn & TensorFlow. O'Reilly Media, Inc., 2017.

ALAMMAR, Jay; GROOTENDORST, Maarten. Hands-on large language models. O'Reilly Media, Inc., 2024.

RAVICHANDIRAN, Sudharsan. Getting started with Google BERT. Packt Publishing, 2021.

HUYEN, Chip. Designing machine learning systems. O'Reilly Media, Inc., 2022.

ABHISHEK, Kumar; ABDELAZIZ, Mounir. Machine learning for imbalanced data. Packt Publishing, 2023.

REIMERS, Nils; GUREVYCH, Iryna. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In: CONFERENCE ON EMPIRICAL METHODS IN NATURAL LANGUAGE PROCESSING, 2019, Association for Computational Linguistics. Proceedings. Disponível em: <https://arxiv.org/abs/1908.10084>. Acesso em: 01 ago. 2024.

NG, Andrew. Bridging AI's proof-of-concept to production gap. HAI Seminar, 22 set. 2020. Video, 1:02:07. Disponível em: <https://www.youtube.com/watch?v=tsPuVAMaADY>.

OPENAI. ChatGPT. 2024. Disponível em: <https://openai.com/chatgpt>. Acesso em: 03 out. 2024.

GOOGLE. BERT: Pre-training of deep bidirectional transformers for language understanding. 2018. Disponível em: <https://github.com/google-research/bert>. Acesso em: 03 out. 2024.